Книг ата за sdev

Пълно ръководство за програмния език sdev

Първо издание · 2026

Съдържание

Съдържание **3**

Книгата за sdev **34**

Предговор 34

Какво е ново в това издание 35

Конвенции 35

Как да четете тази книга 36

Част I — Езикът **38**

Книгата за sdev **40**

Предговор 40

Какво е ново в това издание 41

Конвенции 41

Как да четете тази книга 42

Част I — Езикът **44**

Предговор 44

Какво е ново в това издание 45

Конвенции 45

Как да четете тази книга 46

Част I — Езикът **48**

Съдържание 48

1. Въведение 52

1.1 Защо sdev? 53

1.2 Кратка история 53

1.3 За кого е тази книга 54

1.4 Как да използвате книгата 54

1.5 Конвенции 54

1.6 Първа глътка 54

2. Първи стъпки 55

2.1 Инсталация 55

2.2 Вашата първа програма 55

2.3 REPL 56

2.4 Структура на проекта 56

2.5 Стартиране от IDE 56

2.6 Анатомия на програма 56

2.7 Чести грешки за начинаещи 57

3. Променливи и стойности 57

3.1 Forge 57

3.2 Шест примитивни типа 57

3.3 Инспекция на тип 57

3.4 Преобразуване 58

3.5 Константи 58

3.6 Правила за именуване 58
3.7 Обхват 58
3.8 Истинност 58
3.9 Числа 58
3.10 Низове 59
4. Оператори и изрази 59
4.1 Аритметика 59
4.2 Сравнение 59
4.3 Логически 59
4.4 Низови 59
4.5 Принадлежност 60
4.6 Pipeline 60
4.7 Приоритет 60
5. Контролен поток 60
5.1 If/Then/Else 60
5.2 When 61
5.3 While и until 61
5.4 For ... in 61
5.5 Escape и skip 61
5.6 Етикетирани цикли 61
5.7 Match 61
6. Функции 62
6.1 Дефиниция 62

6.2 Аргументи по подразбиране 62
6.3 Variadic 62
6.4 Връщане 62
6.5 Ламбди 62
6.6 Затваряния 62
6.7 Higher-order 63
6.8 Рекурсия 63
6.9 Генератори 63
6.10 Декоратори 63
7. Колекции 63
7.1 Списъци 63
7.2 Речници 64
7.3 Срязване 64
7.4 Map/Filter/Reduce 64
7.5 Множества 64
7.6 Кортежи 64
7.7 Итерация 64
7.8 Производителност 64
8. Низове задълбочено 65
8.1 Литерали 65
8.2 Индексиране 65
8.3 Търсене 65
8.4 Манипулация 65
8.5 Кодиране 65

8.6 Форматиране 66
8.7 Регулярни изрази 66
8.8 Unicode 66
9. Обектно-ориентирано програмиране 66
9.1 Дефиниция на клас 66
9.2 Инстанции 67
9.3 Наследяване 67
9.4 Class методи 67
9.5 Mixins 67
9.6 Претоварване на оператори 67
9.7 Свойства 67
9.8 Абстрактни класове 67
10. Обработка на грешки 68
10.1 Raise 68
10.2 Attempt/Rescue/Finally 68
10.3 Типове грешки 68
10.4 Множество rescue 68
10.5 Re-raise 68
10.6 Stack traces 69
10.7 Asserts 69
10.8 Идиоми 69
11. Конкурентност 69
11.1 Async функции 69

11.2 Dispatch и await 69
11.3 Паралел 70
11.4 Канали 70
11.5 Отказване 70
11.6 Време 70
11.7 Ядро 70
12. Модули и библиотеки 70
12.1 Локални модули 70
12.2 Gist пакети 71
12.3 Версии 71
12.4 Манифести 71
12.5 Авторство 71
12.6 Обхват 71
12.7 Динамично зареждане 71
13. UI инструментариум 71
13.1 Прозорци 72
13.2 Заглавия и текст 72
13.3 Бутони 72
13.4 Полета 72
13.5 Многоредови полета 72
13.6 Чекбоксове и радиа 72
13.7 Слайдъри и select 73
13.8 Таблици 73
13.9 Tabs 73

13.10 Менюта 73
13.11 Подредба 73
13.12 Реактивни стойности 73
13.13 Събития 73
13.14 Персонализация 73
13.15 Пълен пример 74
14. Графика и canvas 74
14.1 Canvas 74
14.2 Примитиви 74
14.3 Цветове 74
14.4 Анимация 75
14.5 Изображения 75
14.6 Трансформации 75
14.7 Вход 75
14.8 Подскачаща топка 75
15. Карти и ГИС 75
15.1 Карта 76
15.2 Маркери 76
15.3 Tile слоеве 76
15.4 Полилинии 76
15.5 Полигони 76
15.6 GeoJSON 76
15.7 Heatmaps 76
15.8 Събития 76

15.9 Маршрутиране 77
16. Стандартна библиотека 77
16.1 Таблица 77
16.2 I/O 78
16.3 Математика 79
16.4 Колекции 79
16.5 Низове 79
16.6 Време 79
16.7 Мрежа 79
16.8 Система 79
16.9 Кодиране 79
16.10 Криптография 79
17. Инструменти 80
17.1 Уеб IDE 80
17.2 VS Code разширение 80
17.3 CLI 80
17.4 Bytecode компилатор 80
17.5 sdev асистент 80
17.6 Линтер 81
17.7 Форматер 81
17.8 Дебъгер 81
17.9 Тестове 81
18. Напреднали теми 81

18.1 JavaScript Interop 81
18.2 Матрична математика 81
18.3 Буфери и указатели 82
18.4 Виртуално ядро 82
18.5 Сметоизхвърляне 82
18.6 Bytecode VM 82
18.7 Многоезичен код 82
18.8 Вграждане 82
18.9 FFI 82
19. Производителност 83
19.1 Изберете рънтайм 83
19.2 Профилирайте 83
19.3 Избягвайте квадратични шаблони 83
19.4 Кеширане 83
19.5 Преизползвайте буфери 83
19.6 Компилирайте 84
20. Добри практики 84
20.1 Именувайте добре 84
20.2 Предпочитайте глаголи 84
20.3 Малки функции 84
20.4 Прегърнете неизменността 84
20.5 Документирайте намерението 84
20.6 Тествайте рано 85
20.7 Проваляйте се шумно 85

20.8 Композиция над наследяване 85
21. Готварска книга с примери 85
21.1 FizzBuzz 85
21.2 Брояч на думи 86
21.3 Хистограма 86
21.4 Todo приложение 86
21.5 HTTP заявка 86
21.6 Подскачаща топка 86
21.7 Палиндром 87
21.8 Фибоначи 87
21.9 Калкулатор UI 87
21.10 Карта на столици 87
22. Изграждане на реално приложение 88
22.1 Подредба 88
22.2 Storage модул 88
22.3 Render модул 88
22.4 UI 88
22.5 Свързване 89
22.6 Облак 89
22.7 Публикуване 89
23. Често задавани въпроси 89
23.1 Бърз ли е sdev? 89
23.2 Може ли в продукция? 89

23.3 Защо не Python? 90
23.4 Защо два рънтайма? 90
23.5 Бъгове? 90
23.6 Принос? 90
23.7 Запазване между сесии? 90
23.8 Защо "forge"? 90
24. Речник 90
24.1 А–Й 91
24.2 К–Р 91
24.3 С–Я 91
25. Бележки за дизайна 91
25.1 Глаголи над символи 91
25.2 Защо end 92
25.3 Два рънтайма 92
25.4 UI вграден 92
25.5 Многоезичен 92
25.6 Вдъхновения 92
26. Приложения 92
26.1 A. Ключови думи 93
26.2 B. Оператори 93
26.3 C. Вградени функции 93
26.4 D. Изходни езици 93
26.5 E. Bytecode опкодове 93

26.6 F. Syscalls 94
26.7 G. Кодове на грешки 94
26.8 H. CLI флагове 94
26.9 I. Индекс на примерите 94
. 94
. 95

sdev Programming Language 97

Complete Documentation, Tutorial & Reference Guide 97
Table of Contents 97
Introduction 98
Getting Started 99
Running in the IDE 99
Running in the Playground 99
Downloading the Desktop App 100
Your First Program 100
Syntax Overview 101
Comments 101
Blocks 101
No Semicolons Required 102
Indentation 102
Variables & Constants 102
Declaring Variables 102
Reassigning Variables 103

Compound Assignment (shorthand patterns) 103
Built-in Constants 103
Multiple Variables 103
Data Types 104
Numbers 104
Text (Strings) 105
Truth (Booleans) 105
Void (Null) 106
Lists 106
Tomes (Dictionaries) 106
Operators 107
Arithmetic Operators 107
Comparison Operators 107
Logical Operators 108
String & List Concatenation 108
Pipe Operator |> 108
Ternary Operator ~ 109
Operator Precedence (high to low) 109
String Operations 110
Control Flow 111
If / Else — ponder / otherwise 111
While Loop — cycle 112
For-Each Loop — iterate through 112

For-In Loop — within 113
Loop Control 113
Nested Loops 114
Functions 114
Basic Functions — conjure 114
Return Values — yield 114
Default Parameters 115
Lambda Functions (Arrow Syntax) 115
Closures 116
Recursive Functions 117
Higher-Order Functions as Parameters 117
Variadic-style Functions 118
Object-Oriented Programming 118
Defining a Class — essence 118
Creating Instances — new 119
Inheritance — extend 119
Multi-level Inheritance 121
Properties and Computed Properties 122
Error Handling 123
Try / Rescue — attempt / rescue 123
Nested Error Handling 124
Assertion 124
Async & Concurrency 125

Async Functions 125
Await 125
Spawn (Concurrent Execution) 125
Delay 126
Built-in Output Functions 126
Built-in Math Functions 127
Core Math 127
Trigonometry 127
Logarithms & Exponentials 128
Advanced Math 128
Random 128
Built-in String Functions 129
Built-in List Functions 130
Built-in Tome (Dict) Functions 131
Input / Output 131
input(prompt?) — Read User Input 131
print(...) / println(...) — Print (Aliases) 132
Character & Code Point Functions 132
Number Base Conversion 133
Number Formatting & Checking 133
String Checking Functions 134
Regex / Pattern Matching 134
Bitwise Operations 135

Base64 Encoding 136
Hash Function 136
Time & Date 136
Functional Programming 137
Buffer / Memory Operations 138
buffer(size) — Create Byte Buffer 138
Buffer Methods 139
pointer(buffer, offset) — Memory Pointer 139
Pointer Methods 139
Error Handling & Control Flow 140
Additional Aliases 140
Higher-Order Functions 141
each — Transform (Map) 141
sift — Filter 141
fold — Reduce 141
seek — Find First 142
every — All Match 142
some — Any Match 143
enumerate — Index + Value 143
zip — Pair Two Lists 143
Type System & Conversion 143
Getting Type — essence() 144
Converting Types — morph() 144

Type Checking Helpers 144
Collections 144
Set 144
Map 145
Queue (FIFO) 146
Stack (LIFO) 146
LinkedList 147
Matrix Operations 148
Creating Matrices 148
Matrix Arithmetic 148
Vector Operations 149
Matrix Utilities 149
Neural Network Example 150
Graphics & Game Development 150
Canvas Setup 151
Fill and Stroke 151
Basic Shapes 151
Text Drawing 152
Gradients 153
Shadows 153
Transformations 153
Path Drawing 153
Turtle Graphics 154

Sprites & Game Objects 156
Color Functions 156
Graphics Math 157
Complete 2D Graphics Example — Starfield 157
File I/O 158
Reading Files 158
Writing Files 159
Append to File 159
File Utilities 159
Networking 160
HTTP GET 160
HTTP POST 160
Examples & Recipes 160
FizzBuzz 160
Fibonacci (Memoized) 161
Binary Search 161
Quicksort 162
Stack-Based Calculator 162
Caesar Cipher 163
Word Frequency Counter 164
Prime Sieve 164
Linked-List Graph Traversal (BFS) 165
Turtle Mandala 166

JavaScript Interop (JS Interpreter Only) 167
Three Syntax Forms 167
Basic Examples 167
Object Literals 168
Arrow Functions 168
Multi-line JS Blocks 169
Leaflet Map Integration 169
Complete Reference Card 170
Keywords 170
Special Symbols 170
Built-in Constants 171
Quick Reference Examples 171

Част II — UI инструментариум 175

Глава U1 — Прозорци в дълбочина 175
Размери 175
Няколко прозореца 176
Затваряне на прозорец от код 176
Персонализация на прозорец 176
Глава U2 — Бутони в дълбочина 177
Варианти 177
Тома за персонализация 178
Иконки 178
Деактивиране 179

Глава U3 — Текстови полета и форми 179
input — едноредово 179
textarea — многоредово 180
checkbox 180
slider и select 180
Глава U4 — Оформления 181
Редове и колони 181
Групи 181
Табове 181
Таблици 182
Глава U5 — Реактивно състояние с uiget / uiset . 182
Глава U6 — Справочник за персонализация . . . 183
Глава U7 — Облачно работно пространство и автоматична синхронизация 184
Как работи автоматичната синхронизация 184
Ръчно записване 185
Офлайн режим 185
Вложени папки 185

Справочни карти за компоненти 188

audio 188
button 188
canvas 189
checkbox 189

column 189
confirm 190
divider 190
endwindow 190
group 191
heading 191
image 191
input 192
label 192
map 192
menu 193
paragraph 193
progress 193
prompt 194
radio 194
row 195
select 195
slider 195
spacer 196
switch 196
table 196
tabs 197

textarea 197
toast 197
uiget 198
uiset 198
video 198
window 199

Част III — Готварската книга 201

Урок 1 — Здравей, прозорец 201
Стъпка 1 — Празният прозорец 201
Стъпка 2 — Малко съдържание 201
Стъпка 3 — Бутон, който прави нещо 202
Стъпка 4 — Покажете брояча вътре в прозореца 202
Какво научихте 202
Урок 2 — Калкулатор за бакшиш 203
Урок 3 — Списък със задачи 203
Урок 4 — Приложение за бележки 205
Урок 5 — Скицник 206
Урок 6 — Помодоро таймер 206
Урок 7 — Преглед на Markdown 208
Урок 8 — Табло за времето 208
Урок 9 — Карта-изследовател 209
Урок 10 — Мини електронна таблица 210

Урок 11 — Прозорец за чат 211
Урок 12 — Файлов мениджър 212
Урок 13 — Калкулатор 213
Урок 14 — Подскачаща топка 213
Урок 15 — Вашата първа библиотека 214
Стъпка 1 — Напишете библиотеката 214
Стъпка 2 — Публикувайте като публичен Gist . . 215
Стъпка 3 — Използвайте я 215
Накъде нататък 215

Приложение A — Карти и GIS (Leaflet) 217

sdev Leaflet Documentation 219
Interactive Maps for sdev 219
Table of Contents 219
Getting Started 220
Basic Setup 220
HTML Setup 220
Map Creation 221
createMap(containerId, lat, lng, zoom) 221
setMapView(map, lat, lng, zoom) 221
getMapCenter(map) 221
getMapZoom(map) 222
getMapBounds(map) 222

Markers & Popups 222
addMarker(map, lat, lng, popupText?) 222
addMarkerIcon(map, lat, lng, iconUrl, iconSize, popupText?) 223
removeMarker(map, marker) 223
setMarkerPosition(marker, lat, lng) 223
bindPopup(marker, content) 224
bindTooltip(marker, content) 224
openPopup(marker) 224
Shapes 225
addCircle(map, lat, lng, radius, options?) 225
addCircleMarker(map, lat, lng, radius, options?) . . 225
addRectangle(map, lat1, lng1, lat2, lng2, options?) 226
Polylines & Polygons 226
addPolyline(map, points, options?) 226
addPolygon(map, points, options?) 227
addMultiPolygon(map, polygons, options?) 227
Layers 228
addTileLayer(map, urlTemplate, options?) 228
createLayerGroup() 228
addLayerToMap(map, layer) 228
removeLayerFromMap(map, layer) 229
clearLayer(layer) 229
Events 229

onMapClick(map, callback) 229
onMapZoom(map, callback) 230
onMapMove(map, callback) 230
onMarkerClick(marker, callback) 230
onMarkerDrag(marker, callback) 230
Controls 231
addZoomControl(map, position?) 231
addScaleControl(map, options?) 231
addAttributionControl(map, prefix?) 231
addLayerControl(map, baseLayers, overlays) 232
GeoJSON 232
addGeoJSON(map, geoJsonData, options?) 232
geoJSONStyle(feature) 233
Utilities 234
latLng(lat, lng) 234
distance(lat1, lng1, lat2, lng2) 234
boundsContains(bounds, lat, lng) 234
fitBounds(map, lat1, lng1, lat2, lng2) 234
invalidateSize(map) 235
Navigation & Animation 235
panTo(map, lat, lng, options?) 235
panBy(map, x, y) 235
flyTo(map, lat, lng, zoom, options?) 235

flyToBounds(map, lat1, lng1, lat2, lng2, options?) . 235
zoomIn(map, delta?) 236
zoomOut(map, delta?) 236
setZoom(map, zoom) 236
setMinZoom(map, zoom) / setMaxZoom(map, zoom) 236
getMinZoom(map) / getMaxZoom(map) 237
setMaxBounds(map, lat1, lng1, lat2, lng2) 237
Popup & Tooltip Control 237
closePopup(marker) 237
openTooltip(marker) / closeTooltip(marker) 237
setPopupContent(marker, content) 238
setTooltipContent(marker, content) 238
Layer Styling 238
setMarkerIcon(marker, iconUrl, iconSize) 238
setMarkerOpacity(marker, opacity) 238
setMarkerZIndex(marker, zIndex) 238
setCircleRadius(circle, radius) 239
setCircleStyle(circle, options) / setPolylineStyle / setPolygonStyle 239
getPolylineLatLngs(polyline) 239
setPolylineLatLngs(polyline, points) 239
addLatLng(polyline, lat, lng) 240
bringToFront(layer) / bringToBack(layer) 240
Additional Events 240

onLayerClick(layer, callback) 240
onLayerMouseover(layer, callback) / onLayerMouseout(layer, callback) 240
onMapDoubleClick(map, callback) 241
onMapRightClick(map, callback) 241
onMapMousemove(map, callback) 241
Geolocation 241
locate(map, options?) 242
onLocationFound(map, callback) 242
onLocationError(map, callback) 242
stopLocate(map) 242
Overlays 242
addImageOverlay(map, imageUrl, lat1, lng1, lat2, lng2, options?) 243
addVideoOverlay(map, videoUrl, lat1, lng1, lat2, lng2, options?) 243
setImageOpacity(overlay, opacity) 243
setImageUrl(overlay, url) 243
setBounds(overlay, lat1, lng1, lat2, lng2) 243
Feature Groups 244
createFeatureGroup() 244
addToFeatureGroup(featureGroup, layer) 244
removeFromFeatureGroup(featureGroup, layer) . . 244
getFeatureGroupBounds(featureGroup) 244

fitFeatureGroup(map, featureGroup, options?) . . . 245
eachLayer(layerGroup, callback) 245
getLayers(layerGroup) 245
hasLayer(layerGroup, layer) 245
Custom Markers 246
addDivIcon(map, lat, lng, html, className, size) . . 246
Coordinate Utilities 246
getSize(map) 246
latLngToContainerPoint(map, lat, lng) 246
containerPointToLatLng(map, x, y) 246
wrapLng(lng) 247
wrapLat(lat) 247
degreesToDMS(degrees) 247
DMSToDegrees(d, m, s) 247
metersToPixels(map, meters, lat) 247
pixelsToMeters(map, pixels, lat) 248
GIS Analysis Functions 248
bearing(lat1, lng1, lat2, lng2) 248
midpoint(lat1, lng1, lat2, lng2) 248
destination(lat, lng, bearing, distance) 248
area(points) 249
length(points) 249
centroid(points) 249

isPointInPolygon(lat, lng, points) 249
simplify(points, tolerance) 250
interpolateAlong(points, fraction) 250
Layer Visibility 250
getLayerType(layer) 250
isLayerVisible(layer) 250
showLayer(map, layer) / hideLayer(map, layer) . . 251
toggleLayer(map, layer) 251
Complete Examples 251
Interactive City Markers 251
Route Visualization 252
Heatmap Zones 253
Layer Toggle System 254
Animated Marker 255
Style Reference 256
Common Style Options 256
Icon Options 256
Tips & Best Practices 256
Error Handling 257

Приложение Б — Речник 259

Приложение В — Индекс на уроците 262

Колофон 263

ГЛАВА 01

Книгата за sdev

Книгата за sdev

Пълно ръководство за програмния език sdev — второ издание.

> *„Езикът е занаят. Книгата е работилница.“ — екипът на sdev*

Предговор

Това е **второто, разширено издание** на книгата за sdev. То съществува, защото езикът се разрасна далеч отвъд първото издание: цял UI инструментариум, облачно работно пространство, вложени папки, автоматично запазване, ядро с кооперативни задачи, байт-код виртуална машина, „размит“ преводач на код и картографска система с Leaflet вече са граждани от първи клас.

Книгата е организирана в три части:

1. **Езикът** — оригиналният справочник за sdev, обновен.
2. **Инструментариумът** — задълбочен преглед на всеки UI компонент с пълни

таблици за персонализация, шаблони за оформление и рецепти.

3. **Готварската книга** — петнадесет урока стъпка по стъпка, които изграждат

реални приложения от празен лист до работеща програма.

Пълният справочник за Leaflet/GIS е приложение A. Речник и индекс в края позволяват търсене по ключова дума.

Какво е ново в това издание

| Област | Допълнение | |--------|------------| | UI | `window` / `endwindow`, пълен набор компоненти, оформления, менюта, табове, таблици | | UI | Персонализация: варианти, размери, цветове, отстъпи, радиуси | | Състояние | `uiget` / `uiset` реактивни връзки | | Облак | Автоматична синхронизация на цялото работно пространство, вкл. вложени папки | | Облак | Разрешаване на конфликти, офлайн режим, ръчно „Запис в облака“ | | Файлове | Вложени папки, дърво, drag & drop, преименуване, изтриване | | Инструменти | Самостоятелен JavaScript интерпретатор (`sdev-interpreter.js`) | | Инструменти | Самостоятелен Python интерпретатор (`sdev-interpreter.py`) | | Инструменти | Уеб IDE с команден палет, търсене, настройки, терминал | | Документация | Тази книга — английско и българско издание |

Конвенции

- `моноширинен` шрифт се използва за код, имена на файлове и идентификатори.
- **Удебелено** маркира първата поява на нов термин.
- Така изглежда подсказка:

  Съвет. *Натиснете* ***Ctrl + Enter*** *в IDE, за да изпълните текущия файл.*

А така изглежда предупреждение:

> ***Внимание.*** *Блок* `window(...)` *винаги трябва да се затваря с* `endwindow`*,*
>
> *иначе следващите компоненти ще се рендерират мълчаливо в предходния прозорец.*

Как да четете тази книга

Ако вече сте програмирали, прегледайте набързо Част I и преминете към Част II. Ако sdev е първият ви език, четете Част I по ред — всяка глава надгражда предишната. Уроците в Част III са замислени да се пишат на ръка; устоявайте на изкушението да копирате — мускулната памет е половината от урока.

ГЛАВА 02

Част I — Езикът

Част I — Езикът

Тази част е каноничният справочник за sdev. Изчерпателна е: всяка ключова дума, всяка вградена функция, всеки оператор. Ако прочетете само една част, нека е тази.

ГЛАВА 03

Книгата за sdev

Книгата за sdev

Пълно ръководство за програмния език sdev — второ издание.

> *„Езикът е занаят. Книгата е работилница.“ — екипът на sdev*

Предговор

Това е **второто, разширено издание** на книгата за sdev. То съществува, защото езикът се разрасна далеч отвъд първото издание: цял UI инструментариум, облачно работно пространство, вложени папки, автоматично запазване, ядро с кооперативни задачи, байт-код виртуална машина, „размит“ преводач на код и картографска система с Leaflet вече са граждани от първи клас.

Книгата е организирана в три части:

1. **Езикът** — оригиналният справочник за sdev, обновен.
2. **Инструментариумът** — задълбочен преглед на всеки UI компонент с пълни

таблици за персонализация, шаблони за оформление и рецепти.

3. **Готварската книга** — петнадесет урока стъпка по стъпка, които изграждат

реални приложения от празен лист до работеща програма.

Пълният справочник за Leaflet/GIS е приложение A. Речник и индекс в края позволяват търсене по ключова дума.

Какво е ново в това издание

| Област | Допълнение | |--------|------------| | UI | `window` / `endwindow`, пълен набор компоненти, оформления, менюта, табове, таблици | | UI | Персонализация: варианти, размери, цветове, отстъпи, радиуси | | Състояние | `uiget` / `uiset` реактивни връзки | | Облак | Автоматична синхронизация на цялото работно пространство, вкл. вложени папки | | Облак | Разрешаване на конфликти, офлайн режим, ръчно „Запис в облака“ | | Файлове | Вложени папки, дърво, drag & drop, преименуване, изтриване | | Инструменти | Самостоятелен JavaScript интерпретатор (`sdev-interpreter.js`) | | Инструменти | Самостоятелен Python интерпретатор (`sdev-interpreter.py`) | | Инструменти | Уеб IDE с команден палет, търсене, настройки, терминал | | Документация | Тази книга — английско и българско издание |

Конвенции

- `моноширинен` шрифт се използва за код, имена на файлове и идентификатори.
- **Удебелено** маркира първата поява на нов термин.
- Така изглежда подсказка:

  Съвет. *Натиснете* ***Ctrl + Enter*** *в IDE, за да изпълните текущия файл.*

А така изглежда предупреждение:

  Внимание. *Блок* `window(...)` *винаги трябва да се затваря с* `endwindow`*,*

  иначе следващите компоненти ще се рендерират мълчаливо в предходния прозорец.

Как да четете тази книга

Ако вече сте програмирали, прегледайте набързо Част I и преминете към Част II. Ако sdev е първият ви език, четете Част I по ред — всяка глава надгражда предишната. Уроците в Част III са замислени да се пишат на ръка; устоявайте на изкушението да копирате — мускулната памет е половината от урока.

ГЛАВА 04

Част I — Езикът

Част I — Езикът

Тази част е каноничният справочник за sdev. Изчерпателна е: всяка ключова дума, всяка вградена функция, всеки оператор. Ако прочетете само една част, нека е тази.

Предговор

Това е **второто, разширено издание** на книгата за sdev. То съществува, защото езикът се разрасна далеч отвъд първото издание: цял UI инструментариум, облачно работно пространство, вложени папки, автоматично запазване, ядро с кооперативни задачи, байт-код виртуална машина, „размит“ преводач на код и картографска система с Leaflet вече са граждани от първи клас.

Книгата е организирана в три части:

1. **Езикът** — оригиналният справочник за sdev, обновен.
2. **Инструментариумът** — задълбочен преглед на всеки UI компонент с пълни

таблици за персонализация, шаблони за оформление и рецепти.

3. **Готварската книга** — петнадесет урока стъпка по стъпка, които изграждат

реални приложения от празен лист до работеща програма.

Пълният справочник за Leaflet/GIS е приложение A. Речник и индекс в края позволяват търсене по ключова дума.

Какво е ново в това издание

| Област | Допълнение | |--------|------------| | UI | `window` / `endwindow`, пълен набор компоненти, оформления, менюта, табове, таблици | | UI | Персонализация: варианти, размери, цветове, отстъпи, радиуси | | Състояние | `uiget` / `uiset` реактивни връзки | | Облак | Автоматична синхронизация на цялото работно пространство, вкл. вложени папки | | Облак | Разрешаване на конфликти, офлайн режим, ръчно „Запис в облака“ | | Файлове | Вложени папки, дърво, drag & drop, преименуване, изтриване | | Инструменти | Самостоятелен JavaScript интерпретатор (`sdev-interpreter.js`) | | Инструменти | Самостоятелен Python интерпретатор (`sdev-interpreter.py`) | | Инструменти | Уеб IDE с команден палет, търсене, настройки, терминал | | Документация | Тази книга — английско и българско издание |

Конвенции

- `моноширинен` шрифт се използва за код, имена на файлове и идентификатори.
- **Удебелено** маркира първата поява на нов термин.
- Така изглежда подсказка:

  Съвет. *Натиснете* ***Ctrl + Enter*** *в IDE, за да изпълните текущия файл.*

А така изглежда предупреждение:

  Внимание. *Блок* `window(...)` *винаги трябва да се затваря с* `endwindow`*,*

  иначе следващите компоненти ще се рендерират мълчаливо в предходния прозорец.

Как да четете тази книга

Ако вече сте програмирали, прегледайте набързо Част I и преминете към Част II. Ако sdev е първият ви език, четете Част I по ред — всяка глава надгражда предишната. Уроците в Част III са замислени да се пишат на ръка; устоявайте на изкушението да копирате — мускулната памет е половината от урока.

ГЛАВА 05

Част I — Езикът

Част I — Езикът

Тази част е каноничният справочник за sdev. Изчерпателна е: всяка ключова дума, всяка вградена функция, всеки оператор. Ако прочетете само една част, нека е тази.

Съдържание

1. Въведение

- 1.1 Защо sdev? - 1.2 Кратка история - 1.3 За кого е тази книга - 1.4 Как да използвате книгата - 1.5 Конвенции - 1.6 Първа глътка

2. Първи стъпки

- 2.1 Инсталация - 2.2 Вашата първа програма - 2.3 REPL - 2.4 Структура на проекта - 2.5 Стартиране от IDE - 2.6 Анатомия на програма - 2.7 Чести грешки за начинаещи

3. Променливи и стойности

- 3.1 Forge - 3.2 Шест примитивни типа - 3.3 Инспекция на тип - 3.4 Преобразуване - 3.5 Константи - 3.6 Правила за именуване - 3.7 Обхват - 3.8 Истинност - 3.9 Числа - 3.10 Низове

4. Оператори и изрази

- 4.1 Аритметика - 4.2 Сравнение - 4.3 Логически - 4.4 Низови - 4.5 Принадлежност - 4.6 Pipeline - 4.7 Приоритет

5. Контролен поток

- 5.1 If/Then/Else - 5.2 When - 5.3 While и until - 5.4 For ... in - 5.5 Escape и skip - 5.6 Етикетирани цикли - 5.7 Match

6. Функции

- 6.1 Дефиниция - 6.2 Аргументи по подразбиране - 6.3 Variadic - 6.4 Връщане - 6.5 Ламбди - 6.6 Затваряния - 6.7 Higher-order - 6.8 Рекурсия - 6.9 Генератори - 6.10 Декоратори

7. Колекции

- 7.1 Списъци - 7.2 Речници - 7.3 Срязване - 7.4 Map/Filter/Reduce - 7.5 Множества - 7.6 Кортежи - 7.7 Итерация - 7.8 Производителност

8. Низове задълбочено

- 8.1 Литерали - 8.2 Индексиране - 8.3 Търсене - 8.4 Манипулация - 8.5 Кодиране - 8.6 Форматиране - 8.7 Регулярни изрази - 8.8 Unicode

9. Обектно-ориентирано програмиране

- 9.1 Дефиниция на клас - 9.2 Инстанции - 9.3 Наследяване - 9.4 Class методи - 9.5 Mixins - 9.6 Претоварване на оператори - 9.7 Свойства - 9.8 Абстрактни класове

10. Обработка на грешки

- 10.1 Raise - 10.2 Attempt/Rescue/Finally - 10.3 Типове грешки - 10.4 Множество rescue - 10.5 Re-raise - 10.6 Stack traces - 10.7 Asserts - 10.8 Идиоми

11. Конкурентност

- 11.1 Async функции - 11.2 Dispatch и await - 11.3 Паралел - 11.4 Канали - 11.5 Отказване - 11.6 Време - 11.7 Ядро

12. Модули и библиотеки

- 12.1 Локални модули - 12.2 Gist пакети - 12.3 Версии - 12.4 Манифести - 12.5 Авторство - 12.6 Обхват - 12.7 Динамично зареждане

13. UI инструментариум

- 13.1 Прозорци - 13.2 Заглавия и текст - 13.3 Бутони - 13.4 Полета - 13.5 Многоредови полета - 13.6 Чекбоксове и радиа - 13.7 Слайдъри и select - 13.8 Таблици - 13.9 Tabs - 13.10 Менюта - 13.11 Подредба - 13.12 Реактивни стойности - 13.13 Събития - 13.14 Персонализация - 13.15 Пълен пример

14. Графика и canvas

- 14.1 Canvas - 14.2 Примитиви - 14.3 Цветове - 14.4 Анимация - 14.5 Изображения - 14.6 Трансформации - 14.7 Вход - 14.8 Подскачаща топка

15. Карти и ГИС

- 15.1 Карта - 15.2 Маркери - 15.3 Tile слоеве - 15.4 Полилинии - 15.5 Полигони - 15.6 GeoJSON - 15.7 Heatmaps - 15.8 Събития - 15.9 Маршрутиране

16. Стандартна библиотека

- 16.1 Таблица - 16.2 I/O - 16.3 Математика - 16.4 Колекции - 16.5 Низове - 16.6 Време - 16.7 Мрежа - 16.8 Система - 16.9 Кодиране - 16.10 Криптография

17. Инструменти

- 17.1 Уеб IDE - 17.2 VS Code разширение - 17.3 CLI - 17.4 Bytecode компилатор - 17.5 sdev асистент - 17.6 Линтер - 17.7 Форматер - 17.8 Дебъгер - 17.9 Тестове

18. Напреднали теми

- 18.1 JavaScript Interop - 18.2 Матрична математика - 18.3 Буфери и указатели - 18.4 Виртуално ядро - 18.5 Сметоизхвърляне - 18.6 Bytecode VM - 18.7 Многоезичен код - 18.8 Вграждане - 18.9 FFI

19. Производителност

- 19.1 Изберете рънтайм - 19.2 Профилирайте - 19.3 Избягвайте квадратични шаблони - 19.4 Кеширане - 19.5 Преизползвайте буфери - 19.6 Компилирайте

20. Добри практики

- 20.1 Именувайте добре - 20.2 Предпочитайте глаголи - 20.3 Малки функции - 20.4 Прегърнете неизменността - 20.5 Документирайте намерението - 20.6 Тествайте рано - 20.7 Проваляйте се шумно - 20.8 Композиция над наследяване

21. Готварска книга с примери

- 21.1 FizzBuzz - 21.2 Брояч на думи - 21.3 Хистограма - 21.4 Todo приложение - 21.5 HTTP заявка - 21.6 Подскачаща топка - 21.7 Палиндром - 21.8 Фибоначи - 21.9 Калкулатор UI - 21.10 Карта на

столици

22. Изграждане на реално приложение

- 22.1 Подредба - 22.2 Storage модул - 22.3 Render модул - 22.4 UI - 22.5 Свързване - 22.6 Облак - 22.7 Публикуване

23. Често задавани въпроси

- 23.1 Бърз ли е sdev? - 23.2 Може ли в продукция? - 23.3 Защо не Python? - 23.4 Защо два рънтайма? - 23.5 Бъгове? - 23.6 Принос? - 23.7 Запазване между сесии? - 23.8 Защо "forge"?

24. Речник

- 24.1 А–Й - 24.2 К–Р - 24.3 С–Я

25. Бележки за дизайна

- 25.1 Глаголи над символи - 25.2 Защо CODE0 - 25.3 Два рънтайма - 25.4 UI вграден - 25.5 Многоезичен - 25.6 Вдъхновения

26. Приложения

- 26.1 A. Ключови думи - 26.2 B. Оператори - 26.3 C. Вградени функции - 26.4 D. Изходни езици - 26.5 E. Bytecode опкодове - 26.6 F. Syscalls - 26.7 G. Кодове на грешки - 26.8 H. CLI флагове - 26.9 I. Индекс на примерите

1. Въведение

Добре дошли в **Книгата за sdev** — окончателният, изчерпателен наръчник за програмния език sdev. sdev е модерен, изразителен език с уникално четим синтаксис (`forge`, `be`, `speak`, `conjure`, `summon`), създаден да направи кода да изглежда като проза, без да жертва мощта, скоростта или изразителността.

Тази книга е *дълга*. Това е нарочно. Програмните езици са богати светове, а sdev — с четимия си синтаксис, два различни рънтайма, UI инструментариум, ядро, байткод VM и поддръжка на 26 езика — има повече повърхност от повечето.

1.1 Защо sdev?

sdev е създаден, за да преодолее пропастта между *говоренето за код* и *писането на код*. Класическите езици използват алгебрични съкращения (`x = 1`, `def`, `var`). sdev ги заменя с глаголи и съществителни, които носят смисъл: `forge x be 1`. Резултатът е код, който се чете като рецепта или есе.

sdev също приема сериозно идеята, че **един език трябва да идва с всичко необходимо**: UI инструментариум, графичен canvas, картографски API, планировчик в стил ядро, байткод VM, AI инструменти и преводи на ключовите думи на 26 езика.

1.2 Кратка история

sdev започна като експеримент в *проза-подобно* програмиране. Първият интерпретатор беше 400-редов tree-walker. От там се разрасна в два паралелни рънтайма (JavaScript и Python), байткод компилатор, виртуално ядро и IDE в браузъра.

1.3 За кого е тази книга

Тази книга е за всички: пълни начинаещи, професионални програмисти, преподаватели и дизайнери на езици. Всяка глава започва меко и постепенно усложнява материала.

1.4 Как да използвате книгата

Прочетете я последователно първия път. Всеки пример може да бъде стартиран в **онлайн IDE** на https://s-dev.lovable.app или с офлайн интерпретатори. Пишете примерите — не само ги четете.

1.5 Конвенции

Кодовите блокове използват синтаксиса `sdev`. Изходът е показан в `// =>` коментари. Бележки **СЪВЕТ**, **ВНИМАНИЕ** и **ЗАДЪЛБОЧЕНО** маркират важна информация.

1.6 Първа глътка

```
forge greet be conjure(name): "Здравей, " + name end
speak(greet("свят"))
// =&gt; Здравей, свят
```

2. Първи стъпки

В тази глава ще инсталирате sdev (или ще пропуснете инсталацията, използвайки уеб IDE), ще напишете първата си програма и ще разберете компонентите на sdev проект.

2.1 Инсталация

Има няколко начина за стартиране на sdev:

1. **Уеб IDE** — без инсталация, отворете https://s-dev.lovable.app/ide.
2. **JavaScript интерпретатор** — `node sdev-interpreter.js моят-файл.sdev`.
3. **Python интерпретатор** — `python sdev-interpreter.py моят-файл.sdev`.
4. **VS Code разширение** — `.vsix` от менюто Downloads.
5. **Windows инсталатор** — пакетен инсталатор с миниатюрен редактор.

2.2 Вашата първа програма

```
speak("Здравей, свят!")
```

Стартирайте го. Поздравления — вие сте sdev програмист.

2.3 REPL

Стартирайте REPL с `python sdev-interpreter.py` без аргументи. Въвеждайте изрази и те ще бъдат изчислявани веднага. Команди: `:quit`, `:reset`, `:load`, `:save`, `:type`.

2.4 Структура на проекта

Типичен sdev проект има `main.sdev`, папка `lib/` с модули, папка `assets/`, и `README.md`. Няма задължително оформление.

2.5 Стартиране от IDE

В уеб IDE натиснете `Ctrl+Enter` (или `Cmd+Enter`) за изпълнение. Output показва текста; App панелът показва UI; Canvas показва графика; Map показва Leaflet.

2.6 Анатомия на програма

Един sdev файл е поредица от изрази. Включва декларации, контролни структури, дефиниции на функции и класове, и импорти. Отстъпите са за хората; `end` е за парсера.

2.7 Чести грешки за начинаещи

- Забравяне на `end` след `if`, `while` или `conjure`.
- Използване на `=` вместо `be`. И двете работят; `be` е каноничното.
- Смесване на `yes`/`no` с `true`/`false`. Изберете един стил.

3. Променливи и стойности

В sdev създавате променливи с `forge` и ги обвързвате с `be`. Стойностите са динамично типизирани.

3.1 Forge

```
forge name be "Ада"
forge age be 36
```

Преобвързване без `forge` запазва променливата в първоначалния обхват.

3.2 Шест примитивни типа

`number`, `string`, `boolean` (`yes`/`no`), `void` (`nothing`), `collection`, `function`.

3.3 Инспекция на тип

`essence(value)` връща типа като низ.

3.4 Преобразуване

`morph(value, "type")` преобразува типове.

3.5 Константи

`eternal forge PI be 3.14159` — не може да бъде преобвързано.

3.6 Правила за именуване

Идентификаторите започват с буква или подчертавка. snake_case за променливи, PascalCase за класове.

3.7 Обхват

Лексикален обхват. Функциите образуват затваряния (closures).

3.8 Истинност

Falsy: `no`, `nothing`, `0`, `""`, празна колекция.

3.9 Числа

64-битови с плаваща запетая с бърз път за цели числа. `bigint(n)` за произволна точност.

3.10 Низове

Неизменни Unicode. Тройни кавички запазват редовете. Backtick поддържа `${интерполация}`.

4. Оператори и изрази

sdev поддържа стандартни оператори с английски псевдоними.

4.1 Аритметика

`+`, `-`, `*`, `/`, `%`, `**`, `//`.

4.2 Сравнение

`==`, `!=`, `<`, `>`, `<=`, `>=`. Псевдоними: `equals`, `differs`, `above`, `below`.

4.3 Логически

`and`, `or`, `not` (с късо съединение).

4.4 Низови

Конкатенация с `+`. Повторение с `*`. Интерполация с backtick.

4.5 Принадлежност

`x in coll`.

4.6 Pipeline

`" hi " |> trim |> upper |> speak`.

4.7 Приоритет

От най-стегнат до най-слаб: `**`, унарен, `* / // %`, `+ -`, сравнения, `and`, `or`, `|>`, присвояване.

5. Контролен поток

sdev предлага познати контролни структури с английски привкус.

5.1 If/Then/Else

```
if score above 90 then
  speak("Отлично")
else
  speak("Опитай пак")
end
```

5.2 When

```
when:
  is x below 0 then speak("отрицателно")
  is x equals 0 then speak("нула")
  else speak("положително")
end
```

5.3 While и until

```
while count below 10:
  count be count + 1
end
```

5.4 For ... in

```
for name in ["Ада", "Грейс"]: speak(name) end
```

5.5 Escape и skip

`escape` напуска цикъл; `skip` прескача итерация.

5.6 Етикетирани цикли

`label name:` + `escape name` за вложени цикли.

5.7 Match

Деструктуриране: `forge [a, *rest] be [1,2,3]`.

6. Функции

Функциите са обекти от първи клас.

6.1 Дефиниция

```
conjure greet(name be "непознат"):
  speak("Здравей, " + name)
end
```

6.2 Аргументи по подразбиране

Стойности с `be`. Извикване с ключови думи: `greet(name: "Ада")`.

6.3 Variadic

`*items` и `**opts`.

6.4 Връщане

`return value` за ранен изход.

6.5 Ламбди

`forge double be (n) => n * 2`.

6.6 Затваряния

Функциите улавят външни променливи.

6.7 Higher-order

`each`, `sift`, `fold`, `seek`.

6.8 Рекурсия

Поддържа неограничена рекурсия. `tailcall` модификатор за tail-call оптимизация.

6.9 Генератори

Функции с `yield` връщат итератори.

6.10 Декоратори

`@logged conjure work(): ... end`.

7. Колекции

Списъци и речници.

7.1 Списъци

```
forge nums be [1, 2, 3]
speak(nums[0])
gather(nums, 4)
```

7.2 Речници

```
forge user be {name: "Ада", age: 36}
speak(user.name)
```

7.3 Срязване

`portion(coll, start, end?)`.

7.4 Map/Filter/Reduce

`each`, `sift`, `fold`.

7.5 Множества

`unique(coll)`, `set(coll)`.

7.6 Кортежи

`(a, b)` — неизменни.

7.7 Итерация

`for ... in`, `enumerate`, `zip`.

7.8 Производителност

Списъци: O(1) добавяне. Речници: O(1) търсене. `pluck(coll, 0)` е O(n).

8. Низове задълбочено

Текстът е навсякъде — заслужава отделна глава.

8.1 Литерали

`''`, `""`, `"""..."""`, и backtick за интерполация.

8.2 Индексиране

Низовете са индексируеми. Неизменни.

8.3 Търсене

`contains`, `indexOf`, `regex`.

8.4 Манипулация

`upper`, `lower`, `trim`, `replace`, `shatter`, `weave`.

8.5 Кодиране

`base64encode`, `urlencode`, `hexencode`.

8.6 Форматиране

Backtick интерполация или `format`.

8.7 Регулярни изрази

`regex(pattern, s)` връща обект с `groups`.

8.8 Unicode

Низовете са последователности от Unicode codepoints. `locale("bg_BG")` за локализирани операции.

9. Обектно-ориентирано програмиране

Класове с `essence`, инстанции със `summon`, наследяване с `extends`.

9.1 Дефиниция на клас

```
essence Dog:
  share name
  conjure init(name): me.name be name end
  conjure bark(): speak(me.name + " лае!") end
end
```

9.2 Инстанции

```
forge rex be summon Dog(name: "Рекс")
rex.bark()
```

9.3 Наследяване

`essence Puppy extends Dog: ... end`. `ancestor.method()` извиква базовия.

9.4 Class методи

`static share` и `static conjure`.

9.5 Mixins

`essence Bird weave Flyable, Singable: end`.

9.6 Претоварване на оператори

`__add__`, `__eq__`, `__str__`.

9.7 Свойства

`@property` и `@setter`.

9.8 Абстрактни класове

`abstract conjure name()`.

10. Обработка на грешки

Програмите се провалят. sdev предлага точни инструменти.

10.1 Raise

`raise "Без гориво"` или `raise FuelError(...)`.

10.2 Attempt/Rescue/Finally

```
attempt:
  forge data be readFile("missing.txt")
rescue err:
  speak(err.message)
finally:
  speak("край")
end
```

10.3 Типове грешки

`TypeError`, `ValueError`, `IOError`, `KeyError`, `IndexError`, `NetworkError`, `TimeoutError`.

10.4 Множество rescue

Свързани с тип на грешката.

10.5 Re-raise

`raise` без аргумент в `rescue`.

10.6 Stack traces

Грешките имат `.trace` с рамки.

10.7 Asserts

`ensure(условие, "съобщение")`.

10.8 Идиоми

Валидирайте на границите. Не превръщайте rescue в тих поглъщач.

11. Конкурентност

Кооперативна конкурентност.

11.1 Async функции

`async conjure ...` + `await`.

11.2 Dispatch и await

`forge p be dispatch f()` + `await p`.

11.3 Паралел

`await all([...])`, `await any([...])`, `await race([...])`.

11.4 Канали

`channel()`, `send`, `receive`.

11.5 Отказване

`cancel(p)`, `rescue CancelledError`.

11.6 Време

`await delay(1000)`, `every(ms, fn)`.

11.7 Ядро

Всички задачи живеят във виртуално ядро.

12. Модули и библиотеки

Реалните програми се състоят от много файлове.

12.1 Локални модули

`summon "./utils.sdev" as u`.

12.2 Gist пакети

Декентрализиран регистър чрез GitHub Gists.

12.3 Версии

`summon "gist:abc@v1.2.0/util.sdev"`.

12.4 Манифести

Файл `manifest.sdev` декларира зависимости.

12.5 Авторство

Публични `conjure` и `essence` са експортирани. `share private` скрива.

12.6 Обхват

Всеки модул има собствен глобален обхват.

12.7 Динамично зареждане

`loadModule(url)`.

13. UI инструментариум

Декларативен UI: прозорци, форми, дашбордове.

13.1 Прозорци

```
window("Моето приложение", 480, 600)
  heading("Здравей", 1)
endwindow
```

13.2 Заглавия и текст

`heading(text, level)`, `label(text)`, `paragraph(text)`.

13.3 Бутони

`button("Натисни", () => ..., "primary")`. Варианти: primary, secondary, ghost, destructive, link.

13.4 Полета

`input("име", "плейсхолдър", "kind")`. Видове: text, number, password, email, url.

13.5 Многоредови полета

`textarea("body", "...", 10)`.

13.6 Чекбоксове и радиа

`checkbox(bind, label)`, `radio(bind, value, label)`.

13.7 Слайдъри и select

`slider`, `select`.

13.8 Таблици

`table(["Име","Възраст"], [["Ада",36]])`.

13.9 Tabs

tabs/endtabs + tab/endtab.

13.10 Менюта

menu/endmenu + menuitem.

13.11 Подредба

row/endrow, column/endcolumn, group/endgroup.

13.12 Реактивни стойности

`uiget(name)` и `uiset(name, value)`.

13.13 Събития

Бутоните и менютата приемат callback.

13.14 Персонализация

Дикционар с опции като последен аргумент: размер, икона, URL.

13.15 Пълен пример

```
forge todos be []
window("Todos", 360, 480)
  input("new", "Какво да направя?")
  button("Добави", () =&gt; { gather(todos, uiget("new")); uiset("n
  for t in todos: label("• " + t) end
endwindow
```

14. Графика и canvas

2D рисуване и анимация.

14.1 Canvas

`canvas(width, height)`.

14.2 Примитиви

`fill`, `stroke`, `rect`, `circle`, `line`, `polygon`, `text`.

14.3 Цветове

CSS имена, `#rrggbb`, `rgb()`, `hsl()`, `rgba()`.

14.4 Анимация

`frame(fn)`.

14.5 Изображения

`loadImage`, `drawImage`.

14.6 Трансформации

`translate`, `rotate`, `scale`, `push`, `pop`.

14.7 Вход

`onmouse`, `onkey`.

14.8 Подскачаща топка

```
canvas(400, 400)
forge x be 200; forge y be 200; forge dx be 3; forge dy be 2
frame((t) =&gt; {
  fill("black"); rect(0, 0, 400, 400)
  fill("cyan"); circle(x, y, 20)
  x be x + dx; y be y + dy
})
```

15. Карти и ГИС

Интеграция с Leaflet.

15.1 Карта

`map(lat, lon, zoom)`.

15.2 Маркери

`marker(lat, lon, "етикет")`.

15.3 Tile слоеве

`tiles(url, opts)`. Стилове: osm, dark, satellite.

15.4 Полилинии

`polyline([[lat,lon],...], "color")`.

15.5 Полигони

`polygon([[lat,lon],...], "color")`.

15.6 GeoJSON

`geojson(url)`.

15.7 Heatmaps

`heatmap(points)`, `cluster(points)`.

15.8 Събития

`onclick((lat, lon) => ...)`.

15.9 Маршрутиране

`distance(p1, p2)`, `route(from, to)`.

16. Стандартна библиотека

Справочник.

16.1 Таблица

| Функция | Описание | |---|---| | `speak(value)` | Print a value followed by a newline. | | `whisper(value)` | Print a value with no newline. | | `shout(value)` | Print a value in uppercase, attention-grabbing form. | | `ask(prompt)` | Read a line from the user (CLI) or prompt dialog (web). | | `readFile(path)` | Read the entire contents of a text file as a string. | | `writeFile(path, text)` | Write a string to a file, replacing previous contents. | | `appendFile(path, text)` | Append a string to the end of a file. | | `exists(path)` | Returns yes if the file or folder exists. | | `delete(path)` | Removes a file or folder. | | `listdir(path)` | Returns a collection of names inside a folder. | | `essence(value)` | Returns the runtime type as a string. | | `morph(value, type)` | Convert a value to another type. | | `magnitude(value)` | Length of a string or collection; absolute value of a number. | | `least(...)` | Smallest of

the given values or collection. | | `greatest(...)` | Largest of the given values or collection. | | `root(n, k=2)` | k-th root of n; default square root. | | `ground(n)` | Floor; round toward negative infinity. | | `elevate(n)` | Ceiling; round toward positive infinity. | | `around(n, digits=0)` | Round to a number of decimal digits. | | `random()` | Random float in [0, 1). | | `randint(a, b)` | Random integer in [a, b]. | | `choose(coll)` | Random element from a collection. | | `sin(x) / cos(x) / tan(x)` | Trigonometry in radians. | | `asin/acos/atan/atan2` | Inverse trig. | | `log(x, base=e)` | Logarithm. | | `exp(x)` | Exponential e^x. | | `pi / tau / e` | Math constants. | | `gather(coll, value)` | Append a value to a collection in place. | | `pluck(coll, index)` | Remove and return an element at index. | | `portion(coll, start, end?)` | Slice a collection. | | `each(coll, fn)` | Apply fn to each element. | | `sift(coll, predicate)` | Filter a collection. | | `fold(coll, init, fn)` | Reduce a collection to a single value. | | `seek(coll, predicate)` | First element matching predicate. | | `every(coll, predicate)` | Yes if all elements match. | | `some(coll, predicate)` | Yes if any element matches. | | `zip(a, b, ...)` | Pairs elements from multiple collections. | | `enumerate(coll)` | Pairs of (index, element). | | `sequence(start, end, step=1)` | A range of numbers. | | `sort(coll, key?)` | Sort a collection ascending. |

16.2 I/O

`speak`, `whisper`, `shout`, `ask`, `readFile`, `writeFile`, `appendFile`.

16.3 Математика

`magnitude`, `least`, `greatest`, `root`, `random`, `sin`, `cos`, `pi`.

16.4 Колекции

`gather`, `pluck`, `each`, `sift`, `fold`, `zip`, `enumerate`.

16.5 Низове

`upper`, `lower`, `trim`, `replace`, `shatter`, `weave`.

16.6 Време

`now`, `today`, `delay`, `format`.

16.7 Мрежа

`fetch`, `websocket`, `download`.

16.8 Система

`env`, `argv`, `exit`, `osinfo`.

16.9 Кодиране

`base64encode`, `hexencode`, `jsonencode`.

16.10 Криптография

`hash`, `hmac`, `uuid`, `randomBytes`.

17. Инструменти

Редактори, линтер, дебъгер, компилатор.

17.1 Уеб IDE

Пълнофункционално IDE на https://s-dev.lovable.app/ide.

17.2 VS Code разширение

Изтеглете `.vsix`.

17.3 CLI

Флагове: `--repl`, `--compile`, `--exec`, `--lang`, `--ast`, `--time`.

17.4 Bytecode компилатор

`--compile main.sdev` → `main.sdevc`.

17.5 sdev асистент

Вграден AI чатбот.

17.6 Линтер

`sdev lint file.sdev`.

17.7 Форматер

`sdev format file.sdev`.

17.8 Дебъгер

`breakpoint()` пауза в IDE.

17.9 Тестове

`sdev test` пуска `conjure test_*` функции.

18. Напреднали теми

Функции за опитни потребители.

18.1 JavaScript Interop

`js { console.log("native") }`. Многоредови блокове споделят обхват.

18.2 Матрична математика

`matrix`, `matmul`, `inverse`, `transpose`, `det`.

18.3 Буфери и указатели

`buffer(размер)`, `pointer(стойност)`, `deref(p)`.

18.4 Виртуално ядро

`spawn(fn)`, `schedule()`, `syscall("име", args)`.

18.5 Сметоизхвърляне

Mark-and-sweep GC. Принудете цикъл с `gc()`. `heapinfo()`.

18.6 Bytecode VM

Стек-базирана VM. Опкодове: LOAD_CONST, CALL, JUMP, RETURN, MAKE_FUNCTION.

18.7 Многоезичен код

26 езика. `// lang: bg` директива.

18.8 Вграждане

`import { run } from "sdev-interpreter.js"` в JS, `from sdev_interpreter import run` в Python.

18.9 FFI

`ffi.load("libm.so").func(...)`.

19. Производителност

Как да направите sdev по-бърз, когато е необходимо.

19.1 Изберете рънтайм

Tree-walker: най-бавен. Bytecode: 3-5× по-бърз. JS interop: native.

19.2 Профилирайте

`time { ... }`, `profile { ... }`.

19.3 Избягвайте квадратични шаблони

`pluck(coll, 0)` в цикъл е $O(n^2)$. Използвайте `deque`.

19.4 Кеширане

`@memo` за чисти функции.

19.5 Преизползвайте буфери

Един `buffer(N)` извън цикъла.

19.6 Компилирайте

За дълготрайни процеси, `--compile`.

20. Добри практики

Идиоми и шаблони.

20.1 Именувайте добре

sdev възнаграждава описателните имена.

20.2 Предпочитайте глаголи

Имена на функции = глаголи; имена на класове = съществителни.

20.3 Малки функции

Под 20 реда.

20.4 Прегърнете неизменността

Използвайте `eternal` често.

20.5 Документирайте намерението

Коментарите обясняват *защо*.

20.6 Тествайте рано

Един `expect` на функция.

20.7 Проваляйте се шумно

Валидирайте на границите.

20.8 Композиция над наследяване

Mixins (`weave`) надживяват дълбоки йерархии.

21. Готварска книга с примери

Кратки, пълни програми.

21.1 FizzBuzz

```
for i in sequence(1, 100):
  when:
    is i % 15 == 0 then speak("FizzBuzz")
    is i % 3 == 0 then speak("Fizz")
    is i % 5 == 0 then speak("Buzz")
    else speak(i)
  end
end
```

21.2 Брояч на думи

```
forge text be readFile("essay.txt")
speak(magnitude(shatter(text, " ")))
```

21.3 Хистограма

```
forge counts be {}
for w in shatter(readFile("text.txt"), " "):
  counts[w] be (counts[w] or 0) + 1
end
```

21.4 Todo приложение

```
forge todos be []
window("Todos", 360, 480)
  input("new", "Какво да направя?")
  button("Добави", () =&gt; { gather(todos, uiget("new")); uiset("n
  for t in todos: label(t) end
endwindow
```

21.5 HTTP заявка

```
async conjure main():
  forge res be await fetch("https://api.example.com")
  speak(res.body)
end
await main()
```

21.6 Подскачаща топка

```
canvas(400, 400)
```

```
forge x be 200; forge y be 200; forge dx be 3; forge dy be 2
frame((t) =&gt; { fill("black"); rect(0,0,400,400); fill("cyan"); c
```

21.7 Палиндром

```
conjure palindrome(s):
  forge clean be lower(replace(s, " ", ""))
  return clean equals reverse(clean)
end
```

21.8 Фибоначи

```
@memo
conjure fib(n):
  if n below 2 then return n end
  return fib(n-1) + fib(n-2)
end
```

21.9 Калкулатор UI

```
window("Calc", 280, 320)
  input("a", "Първо", "number")
  input("b", "Второ", "number")
  button("+", () =&gt; uiset("r", uiget("a") + uiget("b")))
  label("= " + (uiget("r") or "?"))
endwindow
```

21.10 Карта на столици

```
map(48.0, 10.0, 4)
for (c, lat, lon) in [("София",42.7,23.3),("Берлин",52.5,13.4)]:
  marker(lat, lon, c)
end
```

22. Изграждане на реално приложение

От празен файл до работещ Markdown notebook.

22.1 Подредба

`main.sdev`, `lib/storage.sdev`, `lib/render.sdev`, `ui/notebook.sdev`.

22.2 Storage модул

```
share conjure save(key, text):
  writeFile("notes/" + key + ".md", text)
end
```

22.3 Render модул

Малък регекс конвертор или `summon "gist:abc/markdown.sdev"`.

22.4 UI

```
window("Notebook", 720, 600)
  input("title", "Заглавие")
  textarea("body", "Пишете...", 12)
  button("Запис", () =&gt; storage.save(uiget("title"), uiget("body
endwindow
```

22.5 Свързване

`main.sdev` импортира модулите.

22.6 Облак

Когато сте логнати, работното пространство автоматично синхронизира.

22.7 Публикуване

`sdev publish` качва като gist.

23. Често задавани въпроси

Реални въпроси от реални потребители.

23.1 Бърз ли е sdev?

Сравнима с Python производителност. Bytecode дава 3-5× ускорение.

23.2 Може ли в продукция?

Подходящ за скриптове, образование, прототипи, вътрешни инструменти.

23.3 Защо не Python?

sdev съществува за тези, които искат *по-четим* алтернативен синтаксис.

23.4 Защо два рънтайма?

JavaScript за уеб, Python за научни среди.

23.5 Бъгове?

GitHub issue.

23.6 Принос?

Да, sdev е с отворен код.

23.7 Запазване между сесии?

Влезте в акаунт. IDE автоматично синхронизира.

23.8 Защо "forge"?

Активен глагол; "let" е пасивен.

24. Речник

Бързи дефиниции.

24.1 А–Й

Анонимна функция — без име. **Bytecode** — компактният формат от `--compile`. **Затваряне** — функция, която улавя външни променливи. **Eternal** — модификатор за неизменими обвързвания.

24.2 К–Р

Колекция — списък или речник. **Lambda** — синоним за анонимна функция. **Memo** — декоратор за кеширане. **Модул** — `.sdev` файл. **Морф** — за конверсия на типове.

24.3 С–Я

Обект — инстанция на клас. **Обещание** — стойност, която ще пристигне по-късно. **Реактивна стойност** — UI-обвързана стойност. **Tail call** — извикване в опашка; оптимизирано. **ВМ** — байткод виртуалната машина.

25. Бележки за дизайна

Защо sdev изглежда така.

25.1 Глаголи над символи

Символите четат бързо за експерти и бавно за нови. Глаголите четат приятно.

25.2 Защо `end`

Значимите отстъпи са крехки в редактори и чатове. `end` оцелява copy-paste.

25.3 Два рънтайма

JavaScript и Python покриват повечето среди.

25.4 UI вграден

Език без бутон е език за уроци.

25.5 Многоезичен

Програмирането не трябва да е блокирано от английски.

25.6 Вдъхновения

Lua, Python, Smalltalk, Ruby, AppleScript.

26. Приложения

26.1 A. Ключови думи

`forge`, `be`, `eternal`, `if`, `then`, `else`, `when`, `is`, `while`, `until`, `for`, `in`, `escape`, `skip`, `conjure`, `yield`, `return`, `essence`, `summon`, `share`, `extends`, `me`, `ancestor`, `attempt`, `rescue`, `finally`, `raise`, `dispatch`, `await`, `js`, `end`, `window`, `endwindow`, `row`, `endrow`, `column`, `endcolumn`, `tabs`, `endtabs`, `tab`, `endtab`, `group`, `endgroup`, `menu`, `endmenu`, `menuitem`

26.2 B. Оператори

`+ - * / % ** //`, `== != < > <= >=`, `and or not`, `equals differs above below`, `in`, `=>`, `|>`.

26.3 C. Вградени функции

Виж глава "Стандартна библиотека".

26.4 D. Изходни езици

26 езика, включително български, английски, испански, френски, немски, руски, китайски, японски, корейски и арабски.

26.5 E. Bytecode опкодове

LOAD_CONST, LOAD_NAME, STORE_NAME, CALL, JUMP, JUMP_IF_FALSE, RETURN, MAKE_FUNCTION, BUILD_LIST, BUILD_DICT, IMPORT, RAISE, YIELD, AWAIT, SPAWN, HALT.

26.6 F. Syscalls

open, close, read, write, sleep, fork, socket, connect, time, gc.

26.7 G. Кодове на грешки

E_TYPE, E_VALUE, E_INDEX, E_KEY, E_IO, E_NET, E_TIMEOUT, E_SYNTAX.

26.8 H. CLI флагове

--repl, --compile, --exec, --lang, --ast, --time, --profile, --lint, --format, --test.

26.9 I. Индекс на примерите

Подскачаща топка, Калкулатор, Карта на столици, FizzBuzz, Хистограма, Notebook, Палиндром, Todo, Web scraper, Брояч на думи.

Край на книгата. Версия 2.0. © sdev език.

Допълнение към Част I — Оригинален английски

справочник

За пълнота тук е приложен оригиналният английски справочник, тъй като той съдържа последните технически детайли (синтаксис на оператори, имена на вградени функции, които не се превеждат).

ГЛАВА 06

sdev Programming Language

sdev Programming Language

Complete Documentation, Tutorial & Reference Guide

Table of Contents

1. Introduction
2. Getting Started
3. Syntax Overview
4. Variables & Constants
5. Data Types
6. Operators
7. String Operations
8. Control Flow
9. Functions
10. Object-Oriented Programming
11. Error Handling
12. Async & Concurrency
13. Built-in Output Functions
14. Built-in Math Functions

15. Built-in String Functions
16. Built-in List Functions
17. Built-in Tome (Dict) Functions
18. Higher-Order Functions
19. Type System & Conversion
20. Collections: Set, Map, Queue, Stack, LinkedList
21. Matrix Operations
22. Graphics & Game Development
23. File I/O
24. Networking
25. Examples & Recipes
26. JavaScript Interop
27. Complete Reference Card

Introduction

sdev is an expressive, full-featured programming language with a deliberately unique syntax designed to feel fresh and readable. It supports:

- **Unique English-inspired keywords** — `forge`, `conjure`, `ponder`, `cycle`, `iterate`, `yield`, `yeet`, `skip`
- **Block delimiters** — `::` to open a block and `;;` to close it — no curly braces
- **First-class functions and lambdas** — `(x) -> x * 2`
- **Pipe operator** — `|>` for clean functional composition

- **Full OOP** — `essence` (classes), `extend` (inheritance), `self`, `super`, `new`
- **Error handling** — `attempt :: ... ;; rescue err :: ... ;;`
- **Built-in data structures** — Set, Map, Queue, Stack, LinkedList
- **Matrix math** — `matmul`, `transpose`, `dot`, `reshape`, etc.
- **2D Graphics & Turtle API** — canvas drawing, shapes, gradients, turtle graphics, sprites
- **Async / concurrency** — `async conjure`, `await`, `spawn`
- **JavaScript interop** — `js` keyword for seamless JS integration

sdev runs in the browser-based IDE (at `/ide`), in the Playground (`/`), and as a downloadable Electron desktop application.

Getting Started

Running in the IDE

Open the sdev IDE at `/ide`. Press **Ctrl+Enter** (or the **Run** button) to execute your program. Output appears in the **OUTPUT** panel at the bottom.

Running in the Playground

Go to the main Playground at `/`. Type or paste sdev code in the editor, then click **Run**.

Downloading the Desktop App

In the IDE, go to the **Download** button → **Electron Desktop App**. This gives you three files:

```
package.json
main.js
README.md
```

Install and run:

```
npm install
npm start
```

This opens sdev IDE as a native desktop window that wraps the full web IDE.

Your First Program

```
// hello.sdev
speak("Hello, World!")
```

Output:

```
Hello, World!
```

A slightly more involved first program:

```
forge name be "Alice"
forge age be 30

speak("Name:", name)
```

```
speak("Age:", age)
speak("In 10 years:", age + 10)

conjure greet(person, greeting) ::
  yield greeting + ", " + person + "!"
;;

speak(greet(name, "Welcome"))
```

Output:

```
Name: Alice
Age: 30
In 10 years: 40
Welcome, Alice!
```

Syntax Overview

Comments

```
// This is a single-line comment
# This is also a single-line comment (Python style)
```

Blocks

sdev uses `::` to start a block and `;;` to end it — never curly braces `{}`:

```
ponder x &gt; 5 ::
  speak("x is big")
;;

conjure add(a, b) ::
```

```
  yield a + b
;;
```

Blocks can also be written inline for short single-statement bodies:

```
ponder x &gt; 5 :: speak("big") ;;
```

No Semicolons Required

Statements are separated by newlines. You do NOT need `;` at the end of lines.

```
forge a be 1
forge b be 2
forge c be a + b
speak(c)
```

Indentation

Indentation is cosmetic but strongly recommended. The language uses `::` and `;;` for block structure, not whitespace.

Variables & Constants

Declaring Variables

Use `forge` to declare a new variable and `be` to assign its value:

```
forge name be "Alice"
forge age be 25
forge isActive be yep
forge score be 0.0
```

Reassigning Variables

Just use `be` without `forge`:

```
forge count be 0
count be count + 1
count be 10
```

Compound Assignment (shorthand patterns)

```
forge x be 10
x be x + 5    // x = 15
x be x * 2    // x = 30
x be x - 10   // x = 20
x be x / 4    // x = 5
```

Built-in Constants

sdev provides these constants directly:

```
speak(PI)        // 3.141592653589793
speak(TAU)       // 6.283185307179586 (2 * PI)
speak(E)         // 2.718281828459045
speak(INFINITY)  // Infinity
```

Multiple Variables

```
forge x be 10
forge y be 20
forge z be x + y

// Swap pattern
forge temp be x
x be y
y be temp
```

Data Types

sdev has six core types:

| sdev Type | Description | Literal Example |
|----------|----------|----------| | `number` | Integer or float | `42`, `3.14`, `-7`, `1.5e10` | | `text` | Strings | `"hello"`, `'world'` | | `truth` | Booleans | `yep`, `nope` | | `void` | Null / absent | `void` | | `list` | Arrays | `[1, 2, 3]` | | `tome` | Dictionaries | `:: "key": "value" ;;` |

Functions and class instances are also first-class values.

Numbers

```
forge integer be 42
forge float be 3.14159
forge negative be -100
forge scientific be 1.5e10      // 15,000,000,000
forge hex be 0xFF               // 255
```

Arithmetic is standard, with `^` for power:

```
speak(2 ^ 10)   // 1024
```

```
speak(10 % 3)   // 1
speak(7 / 2)    // 3.5
```

Text (Strings)

```
forge single be 'Hello'
forge double be "World"

// Escape sequences
forge tab be "col1\tcol2"
forge newline be "line1\nline2"
forge quoted be "She said \"hi\""

// String concatenation
forge full be "Hello" + ", " + "World!"

// String repetition
forge stars be "*" * 10   // "**********"

// Access characters
speak("hello"[0])   // h
speak("hello"[-1])  // o (last character)
```

Truth (Booleans)

```
forge t be yep    // true
forge f be nope   // false

// Boolean expressions
speak(5 &gt; 3)          // yep
speak(5 equals 3)     // nope
speak(yep also nope)  // nope
speak(yep either nope) // yep
speak(isnt nope)       // yep
```

Void (Null)

```
forge nothing be void
ponder nothing equals void ::
  speak("It's void!")
;;
```

Lists

```
forge nums be [1, 2, 3, 4, 5]
forge mixed be [1, "two", yep, void]
forge empty be []
forge nested be [[1, 2], [3, 4], [5, 6]]

// Access by index (0-based)
speak(nums[0])    // 1
speak(nums[2])    // 3
speak(nums[-1])   // 5  (last element)

// Modify element
nums[0] be 100
speak(nums)       // [100, 2, 3, 4, 5]

// List length
speak(measure(nums))   // 5

// Slicing
speak(portion(nums, 1, 3))  // [2, 3]
speak(portion(nums, 2))     // [3, 4, 5]
```

Tomes (Dictionaries)

Tomes are key-value stores. Keys are strings.

```
forge person be ::
  "name": "Alice",
```

```
  "age": 30,
  "active": yep
;;

// Access values - both bracket and dot notation work
speak(person["name"])  // Alice
speak(person.age)      // 30

// Set new or existing keys
person["city"] be "New York"
person.age be 31

// Check if key exists
speak(contains(person, "name"))   // yep
speak(contains(person, "phone"))  // nope

// Get all keys, values, entries
speak(inscriptions(person))  // ["name", "age", "active", "city"]
speak(contents(person))      // ["Alice", 31, yep, "New York"]
speak(entries(person))       // [["name","Alice"], ...]
```

Operators

Arithmetic Operators

Operator	Description	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`10 - 4`	`6`
`*`	Multiplication	`4 * 3`	`12`
`/`	Division	`7 / 2`	`3.5`
`%`	Modulo	`10 % 3`	`1`
`^`	Power	`2 ^ 8`	`256`
`-x`	Negation	`-5`	`-5`

Comparison Operators

| Operator | Description | Example | Result | |----------|-------------|---------|--------| | `equals` | Equal to | `5 equals 5` | `yep` | | `differs` | Not equal | `5 differs 3` | `yep` | | `<>` | Not equal (alt) | `5 <> 3` | `yep` | | `<` | Less than | `3 < 5` | `yep` | | `>` | Greater than | `5 > 3` | `yep` | | `<=` | Less or equal | `3 <= 3` | `yep` | | `>=` | Greater or equal | `5 >= 5` | `yep` |

Logical Operators

| Operator | Meaning | Example | |----------|---------|---------| | `also` | AND | `yep also yep` → `yep` | | `either` | OR | `nope either yep` → `yep` | | `isnt` | NOT | `isnt nope` → `yep` |

Short-circuit evaluation applies: `also` stops at first `nope`, `either` stops at first `yep`.

String & List Concatenation

```
"Hello" + " World"    // "Hello World"
[1, 2] + [3, 4]       // [1, 2, 3, 4]
"ha" * 3              // "hahaha"
[0] * 5               // [0, 0, 0, 0, 0]
```

Pipe Operator `|>`

The pipe operator passes the left-hand value as the first argument to the right-hand function:

```
// Without pipe
forge result be weave(sort(sift(each([1,2,3,4,5], x -&gt; x * 2), x

// With pipe — much cleaner!
```

```
forge result be [1, 2, 3, 4, 5]
  |&gt; each(x -&gt; x * 2)
  |&gt; sift(x -&gt; x &gt; 4)
  |&gt; weave(", ")
speak(result)  // "6, 8, 10"
```

Ternary Operator ~

Inline if/else expression:

```
forge x be 10
forge label be x &gt; 5 ~ "big" : "small"
speak(label)  // "big"

// In function calls
speak(magnitude(-5) &gt; 3 ~ "large magnitude" : "small magnitude"

// Nested ternary
forge grade be 85
forge letter be grade &gt;= 90 ~ "A" : grade &gt;= 80 ~ "B" : grad
speak(letter)  // "B"
```

Operator Precedence (high to low)

1. `^` (power)
2. Unary `-`, `isnt`
3. `*`, `/`, `%`
4. `+`, `-`
5. `<`, `>`, `<=`, `>=`
6. `equals`, `differs`, `<>`
7. `also`
8. `either`
9. `~` (ternary)

10. |> (pipe)

String Operations

sdev has a rich string library. All string functions return new strings — strings are immutable.

```
forge s be "  Hello, World!  "

// Case conversion
speak(upper(s))          // "  HELLO, WORLD!  "
speak(lower(s))          // "  hello, world!  "

// Trimming
speak(trim(s))           // "Hello, World!"

// Length
speak(measure(s))        // 18

// Contains / starts / ends
speak(contains(s, "World"))      // yep
speak(startswith(trim(s), "Hello"))  // yep
speak(endswith(trim(s), "!"))        // yep

// Replace (replaces ALL occurrences)
speak(replace("banana", "a", "o"))   // "bonono"

// Find index
speak(locate("hello world", "world"))  // 6
speak(locate("hello", "xyz"))          // -1

// Split and join
forge parts be shatter("a,b,c,d", ",")
speak(parts)                 // ["a", "b", "c", "d"]
speak(weave(parts, " | "))   // "a | b | c | d"
```

```
// Character list
speak(chars("abc"))   // ["a", "b", "c"]

// Reverse
speak(reverse("hello"))  // "olleh"

// Padding
speak(padLeft("5", 4, "0"))    // "0005"
speak(padRight("hi", 6, "!"))  // "hi!!!!"

// Format strings (use {} as placeholders)
speak(format("Hello, {}! You are {} years old.", "Bob", 25))
// "Hello, Bob! You are 25 years old."

// Repeat
speak(repeat("ab", 3))   // "ababab"

// Substring
speak(snatch("hello world", 6, 11))  // "world"
```

Control Flow

If / Else — `ponder` / `otherwise`

```
forge score be 87

ponder score &gt;= 90 ::
  speak("Grade: A")
;; otherwise ponder score &gt;= 80 ::
  speak("Grade: B")
;; otherwise ponder score &gt;= 70 ::
  speak("Grade: C")
;; otherwise ponder score &gt;= 60 ::
  speak("Grade: D")
;; otherwise ::
```

```
  speak("Grade: F")
;;
```

Inline (for short bodies):

```
ponder x &gt; 0 :: speak("positive") ;; otherwise :: speak("non-pos
```

While Loop — `cycle`

```
forge n be 1
cycle n &lt;= 10 ::
  speak(n)
  n be n + 1
;;

// Infinite loop with break
cycle yep ::
  forge x be randint(1, 10)
  speak("Got:", x)
  ponder x equals 7 ::
    speak("Found 7! Stopping.")
    yeet    // break
  ;;
;;
```

For-Each Loop — `iterate through`

```
forge fruits be ["apple", "banana", "cherry"]

iterate fruit through fruits ::
  speak(fruit)
;;
// apple
// banana
// cherry
```

For-In Loop — `within`

```
// Iterate over a list
within fruit be ["apple", "banana", "cherry"] ::
  speak(fruit)
;;

// Iterate over a range
within i be sequence(5) ::
  speak(i)   // 0 1 2 3 4
;;

within i be sequence(1, 11) ::
  speak(i)   // 1 2 3 ... 10
;;

within i be sequence(0, 20, 5) ::
  speak(i)   // 0 5 10 15
;;
```

Loop Control

`yeet` — Break

Exit the nearest enclosing loop immediately:

```
within i be sequence(100) ::
  ponder i &gt; 5 ::
    yeet
  ;;
  speak(i)
;;
// 0 1 2 3 4 5
```

`skip` — Continue

Skip the rest of the current iteration and go to the next:

```
within i be sequence(10) ::
  ponder i % 2 equals 0 ::
    skip   // skip even numbers
  ;;
  speak(i)   // prints only odd: 1 3 5 7 9
;;
```

Nested Loops

```
within i be sequence(1, 4) ::
  within j be sequence(1, 4) ::
    speak(i + " x " + j + " = " + morph(i * j, "text"))
  ;;
;;
```

Functions

Basic Functions — `conjure`

```
conjure greet(name) ::
  speak("Hello, " + name + "!")
;;

greet("Alice")
greet("World")
```

Return Values — `yield`

```
conjure add(a, b) ::
  yield a + b
```

```
;;

conjure max(a, b) ::
  ponder a &gt; b :: yield a ;;
  yield b
;;

forge sum be add(10, 20)
speak(sum)         // 30
speak(max(5, 9))   // 9
```

Default Parameters

```
conjure greet(name, greeting) ::
  ponder greeting equals void :: greeting be "Hello" ;;
  yield greeting + ", " + name + "!"
;;

speak(greet("Alice"))           // Hello, Alice!
speak(greet("Bob", "Welcome"))  // Welcome, Bob!
```

Lambda Functions (Arrow Syntax)

Single expression, one parameter:

```
forge double be x -&gt; x * 2
forge square be x -&gt; x ^ 2
forge negate be x -&gt; -x

speak(double(5))   // 10
speak(square(4))   // 16
speak(negate(7))   // -7
```

Multiple parameters (parentheses required):

```
forge add be (a, b) -&gt; a + b
```

```
forge clamp be (v, lo, hi) -&gt; v &lt; lo ~ lo : v &gt; hi ~ hi :

speak(add(3, 4))           // 7
speak(clamp(150, 0, 100))  // 100
```

Multi-statement lambda body with `::` and `;;`:

```
forge process be x -&gt; ::
  forge doubled be x * 2
  forge result be doubled + 10
  yield result
;;

speak(process(5))   // 20
```

Closures

Functions capture variables from their enclosing scope:

```
conjure makeCounter(start) ::
  forge count be start

  conjure increment() ::
    count be count + 1
    yield count
  ;;

  conjure reset() ::
    count be start
  ;;

  yield :: "next": increment, "reset": reset ;;
;;

forge c be makeCounter(0)
speak(c.next())   // 1
speak(c.next())   // 2
```

```
speak(c.next())   // 3
c.reset()
speak(c.next())   // 1
```

Recursive Functions

```
conjure factorial(n) ::
  ponder n &lt;= 1 :: yield 1 ;;
  yield n * factorial(n - 1)
;;

speak(factorial(10))   // 3628800

// Mutual recursion
conjure isEven(n) ::
  ponder n equals 0 :: yield yep ;;
  yield isOdd(n - 1)
;;

conjure isOdd(n) ::
  ponder n equals 0 :: yield nope ;;
  yield isEven(n - 1)
;;

speak(isEven(4))   // yep
speak(isOdd(7))    // yep
```

Higher-Order Functions as Parameters

```
conjure applyTwice(f, x) ::
  yield f(f(x))
;;

forge double be x -&gt; x * 2
speak(applyTwice(double, 3))   // 12

conjure compose(f, g) ::
```

```
  yield x -&gt; f(g(x))
;;

forge addOne be x -&gt; x + 1
forge triple be x -&gt; x * 3
forge addOneThenTriple be compose(triple, addOne)
speak(addOneThenTriple(4))   // 15
```

Variadic-style Functions

Use lists to simulate variadic arguments:

```
conjure sumAll(nums) ::
  yield fold(nums, 0, (acc, x) -&gt; acc + x)
;;

speak(sumAll([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]))   // 55
```

Object-Oriented Programming

sdev has full class-based OOP using `essence` for class definitions, `new` for instantiation, `self` for the instance reference, `super` for parent class access, and `extend` for inheritance.

Defining a Class — `essence`

```
essence Person ::
  conjure init(self, name, age) ::
    self.name be name
```

```
    self.age be age
  ;;

  conjure greet(self) ::
    speak("Hi, I'm " + self.name + " and I'm " + morph(self.age, "
  ;;

  conjure birthday(self) ::
    self.age be self.age + 1
    speak(self.name + " is now " + morph(self.age, "text") + "!")
  ;;

  conjure toString(self) ::
    yield "Person(" + self.name + ", " + morph(self.age, "text") +
  ;;
;;
```

Creating Instances — `new`

```
forge alice be new Person("Alice", 30)
forge bob be new Person("Bob", 25)

alice.greet()   // Hi, I'm Alice and I'm 30 years old.
bob.greet()     // Hi, I'm Bob and I'm 25 years old.

alice.birthday()   // Alice is now 31!
speak(alice.age)   // 31

speak(alice.toString())  // Person(Alice, 31)
```

Inheritance — `extend`

```
essence Animal ::
  conjure init(self, name, sound) ::
    self.name be name
    self.sound be sound
    self.energy be 100
```

```
    ;;

    conjure makeSound(self) ::
      speak(self.name + " says: " + self.sound)
    ;;

    conjure eat(self, food) ::
      self.energy be self.energy + 20
      speak(self.name + " eats " + food)
    ;;

    conjure status(self) ::
      speak(self.name + " | Energy: " + morph(self.energy, "text"))
    ;;
  ;;

  essence Dog extend Animal ::
    conjure init(self, name, breed) ::
      super.init(name, "Woof!")
      self.breed be breed
      self.tricks be []
    ;;

    conjure makeSound(self) ::
      speak(self.name + " barks: WOOF WOOF!")   // Override parent
    ;;

    conjure learnTrick(self, trick) ::
      gather(self.tricks, trick)
      speak(self.name + " learned: " + trick)
    ;;

    conjure showTricks(self) ::
      ponder measure(self.tricks) equals 0 ::
        speak(self.name + " knows no tricks.")
        yield void
      ;;
      speak(self.name + " can do: " + weave(self.tricks, ", "))
    ;;
```

```
;;

forge rex be new Dog("Rex", "German Shepherd")
rex.makeSound()                // Rex barks: WOOF WOOF!
rex.eat("kibble")              // Rex eats kibble
rex.learnTrick("sit")
rex.learnTrick("shake")
rex.learnTrick("roll over")
rex.showTricks()               // Rex can do: sit, shake, roll ove
rex.status()                   // Rex | Energy: 120
```

Multi-level Inheritance

```
essence Vehicle ::
  conjure init(self, brand, speed) ::
    self.brand be brand
    self.speed be speed
    self.fuel be 100
  ;;

  conjure describe(self) ::
    yield self.brand + " going " + morph(self.speed, "text") + " kn
  ;;
;;

essence Car extend Vehicle ::
  conjure init(self, brand, speed, doors) ::
    super.init(brand, speed)
    self.doors be doors
  ;;

  conjure honk(self) ::
    speak(self.brand + ": Beep beep!")
  ;;
;;

essence ElectricCar extend Car ::
  conjure init(self, brand, speed, doors, range) ::
```

```
    super.init(brand, speed, doors)
    self.range be range
    self.battery be 100
  ;;

  conjure charge(self) ::
    self.battery be 100
    speak(self.brand + " fully charged!")
  ;;

  conjure describe(self) ::
    yield super.describe() + " (Electric, " + morph(self.range, "to
  ;;
;;

forge tesla be new ElectricCar("Tesla Model 3", 250, 4, 500)
speak(tesla.describe())   // Tesla Model 3 going 250 km/h (Electric
tesla.honk()              // Tesla Model 3: Beep beep!
tesla.charge()            // Tesla Model 3 fully charged!
```

Properties and Computed Properties

```
essence Circle ::
  conjure init(self, radius) ::
    self.radius be radius
  ;;

  conjure area(self) ::
    yield PI * self.radius ^ 2
  ;;

  conjure circumference(self) ::
    yield 2 * PI * self.radius
  ;;

  conjure scale(self, factor) ::
    self.radius be self.radius * factor
    yield self
```

```
  ;;

  conjure toString(self) ::
    yield "Circle(r=" + morph(nearby(self.radius * 100) / 100, "tex
  ;;
;;

forge c be new Circle(5)
speak(nearby(c.area()))           // 79
speak(nearby(c.circumference()))  // 31

c.scale(2)
speak(c.toString())   // Circle(r=10)
```

Error Handling

Try / Rescue — `attempt` / `rescue`

Use `attempt` to try code that might fail, and `rescue` to catch errors:

```
attempt ::
  forge result be 10 / 0
  speak(result)
;; rescue err ::
  speak("Caught an error: " + err)
;;

attempt ::
  forge nums be [1, 2, 3]
  speak(nums[10])   // index out of bounds
;; rescue err ::
  speak("Error accessing list:", err)
;;
```

Nested Error Handling

```
conjure safeDivide(a, b) ::
  attempt ::
    ponder b equals 0 ::
      speak("Error: division by zero")
      yield void
    ;;
    yield a / b
  ;; rescue err ::
    speak("Unexpected error:", err)
    yield void
  ;;
;;

speak(safeDivide(10, 2))   // 5
speak(safeDivide(10, 0))   // Error: division by zero
```

Assertion

```
conjure validateAge(age) ::
  attempt ::
    ponder age &lt; 0 ::
      speak("Error: Age cannot be negative")
      yield nope
    ;;
    ponder age &gt; 150 ::
      speak("Error: Age is unreasonably large")
      yield nope
    ;;
    yield yep
  ;; rescue err ::
    speak("Validation error:", err)
    yield nope
  ;;
;;
```

Async & Concurrency

Async Functions

Use `async conjure` to define an asynchronous function:

```
async conjure fetchData(url) ::
  forge response be await fetch(url)
  yield response
;;
```

Await

Use `await` inside `async` functions to wait for asynchronous operations:

```
async conjure loadUserProfile(id) ::
  forge user be await fetchData("https://api.example.com/users/" +
  forge posts be await fetchData("https://api.example.com/posts?us

  yield ::
    "user": user,
    "posts": posts
  ;;
;;
```

Spawn (Concurrent Execution)

Use `spawn` to run functions concurrently without waiting for them:

```
conjure worker(name, ms) ::
  delay(ms)
  speak(name + " finished after " + morph(ms, "text") + "ms")
;;

spawn worker("Task A", 2000)
spawn worker("Task B", 500)
spawn worker("Task C", 1000)
// Task B completes first, then C, then A
```

Delay

Pause execution for a number of milliseconds:

```
speak("Starting...")
delay(1000)
speak("One second later...")
delay(2000)
speak("Three seconds total")
```

Built-in Output Functions

| Function | Description | Example |
|---------|----------|---------| | `speak(...)` | Print values with spaces between them | `speak("x =", 42)` → `x = 42` | | `whisper(...)` | Print values concatenated (no spaces) | `whisper("a", "b")` → `ab` | | `shout(...)` | Print values uppercased | `shout("hello")` → `HELLO` |

```
forge x be 42
forge name be "Alice"

speak("The answer is", x)       // The answer is 42
```

```
speak(name, "scored", x, "%")   // Alice scored 42 %
whisper("[", x, "]")             // [42]
shout("warning: low battery")    // WARNING: LOW BATTERY
```

Built-in Math Functions

Core Math

| Function | Description | Example | |---------|----------|---------| | `magnitude(x)` | Absolute value | `magnitude(-5)` → `5` | | `root(x)` | Square root | `root(16)` → `4` | | `ground(x)` | Floor (round down) | `ground(3.7)` → `3` | | `elevate(x)` | Ceiling (round up) | `elevate(3.2)` → `4` | | `nearby(x)` | Round to nearest int | `nearby(3.5)` → `4` | | `least(...)` | Minimum value | `least(3, 1, 4, 1)` → `1` | | `greatest(...)` | Maximum value | `greatest(3, 1, 4, 1)` → `4` |

Trigonometry

| Function | Description | |---------|----------| | `sin(x)` | Sine (radians) | | `cos(x)` | Cosine (radians) | | `tan(x)` | Tangent (radians) | | `asin(x)` | Arcsine | | `acos(x)` | Arccosine | | `atan(x)` | Arctangent | | `atan2(y, x)` | Two-argument arctangent | | `radians(deg)` | Convert degrees to radians | | `degrees(rad)` | Convert radians to degrees |

Logarithms & Exponentials

| Function | Description | |----------|----------| | `log(x)` | Natural logarithm | | `log10(x)` | Base-10 logarithm | | `log2(x)` | Base-2 logarithm | | `exp(x)` | e^x |

Advanced Math

| Function | Description | Example | |----------|----------|---------| | `clamp(v, min, max)` | Constrain value to range | `clamp(150, 0, 100)` → `100` | | `lerp(a, b, t)` | Linear interpolation | `lerp(0, 100, 0.3)` → `30` | | `mapRange(v, fromLo, fromHi, toLo, toHi)` | Map value between ranges | `mapRange(50, 0, 100, 0, 1)` → `0.5` | | `sum(list)` | Sum all elements of a list | `sum([1,2,3,4])` → `10` | | `average(list)` | Mean value of a list | `average([1,2,3,4])` → `2.5` | | `sign(x)` | Sign: -1, 0, or 1 | `sign(-5)` → `-1` | | `dist(x1, y1, x2, y2)` | Distance between two points | `dist(0,0,3,4)` → `5` |

Random

| Function | Description | Example | |----------|----------|---------| | `chaos()` | Random float 0-1 | `chaos()` → `0.7234` | | `randint(min, max)` | Random integer (inclusive) | `randint(1, 6)` → `4` | | `pick(list)` | Random element from list | `pick(["a","b","c"])` → `"b"` | | `shuffle(list)` | Shuffle a copy of list | `shuffle([1,2,3])` → `[3,1,2]` |

```
// Dice rolling
```

```
conjure rollDice(sides) ::
  yield randint(1, sides)
;;

speak("d6:", rollDice(6))
speak("d20:", rollDice(20))

// Coin flip
speak(chaos() &gt; 0.5 ~ "Heads" : "Tails")
```

Built-in String Functions

| Function | Description | Signature |
|---------|----------|----------| | `upper(s)` | Uppercase | `upper("hi")` → `"HI"` | | `lower(s)` | Lowercase | `lower("HI")` → `"hi"` | | `trim(s)` | Remove surrounding whitespace | `trim("  hi  ")` → `"hi"` | | `reverse(s)` | Reverse string | `reverse("abc")` → `"cba"` | | `measure(s)` | String length | `measure("hello")` → `5` | | `contains(s, sub)` | Check substring | `contains("hello", "ell")` → `yep` | | `startswith(s, prefix)` | Starts with | `startswith("hello", "he")` → `yep` | | `endswith(s, suffix)` | Ends with | `endswith("hello", "lo")` → `yep` | | `replace(s, old, new)` | Replace all occurrences | `replace("aaa", "a", "b")` → `"bbb"` | | `locate(s, sub)` | Index of substring (-1 if not found) | `locate("hello", "l")` → `2` | | `shatter(s, sep)` | Split string into list | `shatter("a,b", ",")` → `["a","b"]` | | `chars(s)` | Split into list of characters | `chars("abc")` → `["a","b","c"]` | | `weave(list, sep)` | Join list with separator | `weave(["a","b"], "-")` → `"a-b"` | | `padLeft(s, w, c)` | Pad left to width | `padLeft("5", 3, "0")` → `"005"` | |

`padRight(s, w, c)` | Pad right to width | `padRight("hi", 5, ".")` → `"hi..."` | | `format(s, ...)` | Substitute `{}` placeholders | `format("Hi {}!", "Bob")` → `"Hi Bob!"` | | `repeat(s, n)` | Repeat string n times | `repeat("ab", 3)` → `"ababab"` | | `snatch(s, start, end?)` | Substring by index | `snatch("hello", 1, 3)` → `"el"` |

Built-in List Functions

| Function | Description | Example |
|---------|----------|---------| | `measure(list)` | Length of list | `measure([1,2,3])` → `3` | | `gather(list, item)` | Append item (mutates) | `gather([1,2], 3)` → `[1,2,3]` | | `pluck(list)` | Remove and return last item | `pluck([1,2,3])` → `3` | | `snatch(list, idx)` | Remove item at index | `snatch([1,2,3], 1)` → removes `2` | | `insert(list, idx, item)` | Insert at index | `insert([1,3], 1, 2)` → `[1,2,3]` | | `portion(list, start, end?)` | Slice | `portion([1,2,3,4], 1, 3)` → `[2,3]` | | `reverse(list)` | Return reversed copy | `reverse([1,2,3])` → `[3,2,1]` | | `sort(list)` | Sort ascending copy | `sort([3,1,2])` → `[1,2,3]` | | `sortDesc(list)` | Sort descending copy | `sortDesc([1,3,2])` → `[3,2,1]` | | `unique(list)` | Remove duplicates | `unique([1,1,2,2,3])` → `[1,2,3]` | | `flatten(list)` | Flatten nested lists | `flatten([[1,2],[3,4]])` → `[1,2,3,4]` | | `concat(a, b)` | Concatenate two lists | `concat([1,2],[3,4])` → `[1,2,3,4]` | | `contains(list, item)` | Check membership | `contains([1,2,3], 2)` → `yep` | | `first(list)` | First element

| `first([1,2,3])` → `1` | | `last(list)` | Last element | `last([1,2,3])` → `3` | | `clone(list)` | Deep copy | `clone([1,[2,3]])` |

Built-in Tome (Dict) Functions

| Function | Description | |----------|----------| | `inscriptions(tome)` | Get all keys as a list | | `contents(tome)` | Get all values as a list | | `entries(tome)` | Get `[[key, value], ...]` pairs | | `contains(tome, key)` | Check if key exists | | `merge(t1, t2, ...)` | Merge dicts (later overrides earlier) | | `erase(tome, key)` | Remove key |

```
forge a be :: "x": 1, "y": 2 ;;
forge b be :: "y": 99, "z": 3 ;;
forge c be merge(a, b)
speak(c)  // {"x": 1, "y": 99, "z": 3}
```

Input / Output

`input(prompt?)` — Read User Input

Prompts the user for input and returns the entered text:

```
forge name be input("What is your name? ")
speak("Hello, " + name + "!")
```

```
forge age be morph(input("Enter your age: "), "number")
speak("You are " + age + " years old")
```

`print(...)` / `println(...)` — Print (Aliases)

Standard aliases for `speak()`:

```
print("Hello")
println("World")
```

Character & Code Point Functions

Function	Description	Example
`chr(n)`	Number to character	`chr(65)` → `"A"`
`ord(c)`	Character to number	`ord("A")` → `65`

```
speak(chr(72) + chr(105))  // "Hi"
speak(ord("A"))             // 65

// Build a Caesar cipher
conjure encrypt(text, shift) ::
  forge result be ""
  iterate through text ::
    forge code be ord(item) + shift
    result be result + chr(code)
  ;;
  yield result
;;
speak(encrypt("ABC", 3))  // "DEF"
```

Number Base Conversion

Function	Description	Example
`hex(n)`	Number to hex string	`hex(255)` → `"0xFF"`
`oct(n)`	Number to octal	`oct(8)` → `"0o10"`
`bin(n)`	Number to binary	`bin(10)` → `"0b1010"`
`parseNum(str, base?)`	Parse string with base	`parseNum("FF", 16)` → `255`

```
speak(hex(255))        // "0xFF"
speak(bin(42))         // "0b101010"
speak(oct(64))         // "0o100"
speak(parseNum("1010", 2))  // 10
speak(parseNum("FF", 16))   // 255
```

Number Formatting & Checking

Function	Description	Example
`toFixed(n, digits)`	Format decimal places	`toFixed(3.14159, 2)` → `"3.14"`
`toPrecision(n, p)`	Format to precision	`toPrecision(123.456, 4)` → `"123.5"`
`isNaN(v)`	Check if NaN	`isNaN(0/0)` → `yep`
`isFinite(v)`	Check if finite	`isFinite(INFINITY)` → `nope`
`isInteger(v)`	Check if integer	`isInteger(3.0)` → `yep`

```
forge pi be 3.14159265
speak(toFixed(pi, 2))      // "3.14"
speak(toPrecision(pi, 4))  // "3.142"
```

```
speak(isInteger(42))       // yep
speak(isFinite(1/0))       // nope
```

String Checking Functions

| Function | Description | Example |
|---------|----------|---------| | `capitalize(s)` | First char uppercase | `capitalize("hello")` → `"Hello"` | | `title(s)` | Title Case | `title("hello world")` → `"Hello World"` | | `center(s, width, char?)` | Center-pad | `center("hi", 10, "-")` → `"----hi----"` | | `trimLeft(s)` | Trim left whitespace | `trimLeft("  hi")` → `"hi"` | | `trimRight(s)` | Trim right whitespace | `trimRight("hi  ")` → `"hi"` | | `isUpper(s)` | All uppercase? | `isUpper("ABC")` → `yep` | | `isLower(s)` | All lowercase? | `isLower("abc")` → `yep` | | `isDigit(s)` | All digits? | `isDigit("123")` → `yep` | | `isAlpha(s)` | All alphabetic? | `isAlpha("abc")` → `yep` | | `isAlphaNum(s)` | All alphanumeric? | `isAlphaNum("abc123")` → `yep` | | `isSpace(s)` | All whitespace? | `isSpace(" ")` → `yep` |

```
speak(capitalize("hello world"))  // "Hello world"
speak(title("the quick brown fox"))  // "The Quick Brown Fox"
speak(center("TITLE", 20, "="))  // "=======TITLE========"
speak(isDigit("42"))   // yep
speak(isAlpha("hello"))  // yep
```

Regex / Pattern Matching

| Function | Description | |----------|----------| | `match(text, pattern)` | First regex match (returns list or void) | | `matchAll(text, pattern)` | All regex matches | | `replaceRegex(text, pattern, replacement)` | Regex replace (global) | | `test(text, pattern)` | Test if pattern matches |

```
forge text be "Hello 123 World 456"
speak(match(text, "\\d+"))           // ["123"]
speak(matchAll(text, "\\d+"))        // [["123"], ["456"]]
speak(replaceRegex(text, "\\d+", "#"))  // "Hello # World #"
speak(test(text, "\\d+"))           // yep

// Validate email
conjure isEmail(s) ::
  yield test(s, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$
;;
speak(isEmail("user@example.com"))  // yep
```

Bitwise Operations

| Function | Description | Example | |----------|----------|---------| | `bitAnd(a, b)` | Bitwise AND | `bitAnd(5, 3)` → `1` | | `bitOr(a, b)` | Bitwise OR | `bitOr(5, 3)` → `7` | | `bitXor(a, b)` | Bitwise XOR | `bitXor(5, 3)` → `6` | | `bitNot(a)` | Bitwise NOT | `bitNot(0)` → `-1` | | `bitShiftLeft(a, n)` | Left shift | `bitShiftLeft(1, 3)` → `8` | | `bitShiftRight(a, n)` | Right shift | `bitShiftRight(8, 2)` → `2` |

```
// Flags / bitmask example
forge READ be 1
```

```
forge WRITE be 2
forge EXEC be 4

forge perms be bitOr(READ, WRITE)  // 3
speak(bitAnd(perms, READ) &gt; 0)     // yep (has read)
speak(bitAnd(perms, EXEC) &gt; 0)     // nope (no exec)
```

Base64 Encoding

| Function | Description | |----------|----------| | `base64encode(text)` | Encode text to base64 | | `base64decode(text)` | Decode base64 to text |

```
forge encoded be base64encode("Hello, World!")
speak(encoded)  // "SGVsbG8sIFdvcmxkIQ=="
speak(base64decode(encoded))  // "Hello, World!"
```

Hash Function

```
speak(hash("hello"))     // deterministic 32-bit integer hash
speak(hash([1, 2, 3]))   // works on any value
```

Time & Date

| Function | Description | |----------|----------| | `now()` | Current timestamp in milliseconds | | `timestamp()` | ISO 8601 string | | `time()` | Detailed time tome with year, month, day, etc. | | `formatTime(ms)` | Format milliseconds

to ISO string |

```
forge t be time()
speak("Year: " + t.year)
speak("Month: " + t.month)
speak("Day: " + t.day)
speak("Hour: " + t.hour)
speak("ISO: " + t.iso)

forge start be now()
// ... do work ...
forge elapsed be now() - start
speak("Took " + elapsed + "ms")
```

Functional Programming

| Function | Description | |----------|----------| | `compose(f, g, ...)` | Right-to-left function composition | | `pipe(value, f, g, ...)` | Left-to-right value piping | | `curry(fn, arity)` | Currying | | `memoize(fn)` | Cache function results | | `tap(value, fn)` | Execute fn with value, return value | | `times(n, fn)` | Call fn n times with index | | `groupBy(list, fn)` | Group elements by key function | | `chunk(list, size)` | Split list into chunks |

```
// Compose
conjure double(x) :: yield x * 2 ;;
conjure addOne(x) :: yield x + 1 ;;
forge doubleAndAdd be compose(addOne, double)
speak(doubleAndAdd(5))  // 11

// Pipe
forge result be pipe(5, double, addOne, double)
```

```
speak(result)  // 22

// Memoize (cache expensive computations)
conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  yield fib(n - 1) + fib(n - 2)
;;
forge fastFib be memoize(fib)
speak(fastFib(10))  // 55

// Group and chunk
forge words be ["apple", "avocado", "banana", "blueberry", "cherry
speak(groupBy(words, w -&gt; charAt(w, 0)))
// :: a: ["apple", "avocado"], b: ["banana", "blueberry"], c: ["che

speak(chunk([1,2,3,4,5,6,7], 3))  // [[1,2,3], [4,5,6], [7]]

// Times
speak(times(5, i -&gt; i * i))  // [0, 1, 4, 9, 16]
```

Buffer / Memory Operations

sdev provides low-level byte buffer operations for systems programming:

`buffer(size)` — Create Byte Buffer

```
forge mem be buffer(1024)
mem.set(0, 255)
mem.set(1, 128)
speak(mem.get(0))    // 255
speak(mem.get(1))    // 128
speak(mem.size())    // 1024
```

Buffer Methods

| Method | Description | |--------|----------| | `buf.get(index)` | Read byte at index | | `buf.set(index, value)` | Write byte at index (0-255) | | `buf.fill(value)` | Fill entire buffer | | `buf.slice(start, end)` | Get portion as list | | `buf.toList()` | Convert to list of numbers | | `buf.toText()` | Decode as UTF-8 text | | `buf.fromString(text)` | Write UTF-8 text into buffer | | `buf.copyTo(target)` | Copy data to another buffer | | `buf.size()` | Get buffer size |

`pointer(buffer, offset)` — Memory Pointer

```
forge mem be buffer(256)
forge ptr be pointer(mem, 0)
ptr.write(42)
speak(ptr.read())        // 42

forge ptr2 be ptr.advance(4)
ptr2.writeU16(1024)
speak(ptr2.readU16())    // 1024

ptr2.writeU32(0xDEADBEEF)
speak(hex(ptr2.readU32()))  // "0xDEADBEEF"
```

Pointer Methods

| Method | Description | |--------|----------| | `ptr.read()` | Read byte | | `ptr.write(v)` | Write byte | | `ptr.advance(n?)` | Return new pointer at offset+n | | `ptr.readU16()` | Read 16-bit unsigned (little-endian) | | `ptr.readU32()` | Read 32-bit unsigned (little-endian) | | `ptr.writeU16(v)` | Write

16-bit unsigned | | `ptr.writeU32(v)` | Write 32-bit unsigned |

Error Handling & Control Flow

| Function | Description | |----------|----------| | `exit(code?)` | Terminate program with exit code | | `panic(message)` | Fatal error (kernel panic) | | `throw(message)` | Throw custom error |

```
// Throw and catch
attempt ::
  forge x be input("Enter a number: ")
  forge n be morph(x, "number")
  ponder n &lt; 0 ::
    throw("Negative numbers not allowed!")
  ;;
  speak("Square root: " + root(n))
rescue err ::
  speak("Error: " + err)
;;

// Exit
ponder badCondition ::
  exit(1)
;;
```

Additional Aliases

| Alias | Original | Description | |-------|----------|-------------| | `print()` | `speak()` | Standard print | | `range()` | `sequence()` | Standard range | | `typeof()` | `gettype()` | Type checking | | `sleep()` | `delay()` | Pause (no-op in browser) | | `keys()` | `inscriptions()` | Dict keys | | `values()` | `contents()` | Dict values | | `freeze(obj)` | — | Make object immutable | | `isFrozen(obj)` | — | Check if frozen | | `hash(value)` | — | 32-bit hash code |

Higher-Order Functions

`each` — Transform (Map)

Apply a function to every element and return a new list:

```
forge nums be [1, 2, 3, 4, 5]
speak(each(nums, x -&gt; x ^ 2))          // [1, 4, 9, 16, 25]
speak(each(nums, (x, i) -&gt; x + i))     // [1, 3, 5, 7, 9]  (valu
speak(each(["a","b","c"], upper))       // ["A", "B", "C"]
```

`sift` — Filter

Keep only elements where the predicate returns `yep`:

```
forge nums be [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
speak(sift(nums, x -&gt; x % 2 equals 0))       // [2, 4, 6, 8, 10
speak(sift(nums, x -&gt; x &gt; 5))                // [6, 7, 8, 9,
speak(sift(["", "hi", "", "world"], x -&gt; measure(x) &gt; 0))  /
```

`fold` — Reduce

Reduce a list to a single value using an accumulator:

```
forge nums be [1, 2, 3, 4, 5]
speak(fold(nums, 0, (acc, x) -&gt; acc + x))   // 15 (sum)
speak(fold(nums, 1, (acc, x) -&gt; acc * x))   // 120 (product)

// Build a string
speak(fold(["a","b","c"], "", (acc, x) -&gt; acc + x + "-"))  // "a

// Find max
speak(fold(nums, nums[0], (a, b) -&gt; a &gt; b ~ a : b))   // 5
```

`seek` — Find First

Return the first element matching a predicate, or `void`:

```
forge people be [
  :: "name": "Alice", "age": 22 ;;,
  :: "name": "Bob", "age": 35 ;;,
  :: "name": "Carol", "age": 28 ;;
]

forge adult be seek(people, p -&gt; p.age &gt;= 30)
speak(adult.name)   // Bob

forge nobody be seek(people, p -&gt; p.age &gt; 100)
speak(nobody)   // void
```

`every` — All Match

Returns `yep` if ALL elements satisfy the predicate:

```
speak(every([2,4,6,8], x -&gt; x % 2 equals 0))  // yep
speak(every([1,2,3], x -&gt; x &gt; 0))             // yep
speak(every([1,-2,3], x -&gt; x &gt; 0))            // nope
```

`some` — Any Match

Returns `yep` if ANY element satisfies the predicate:

```
speak(some([1,3,5,6], x -&gt; x % 2 equals 0))  // yep (6 is even)
speak(some([1,3,5,7], x -&gt; x % 2 equals 0))  // nope
```

`enumerate` — Index + Value

Pairs each element with its index:

```
speak(enumerate(["a","b","c"]))
// [[0,"a"], [1,"b"], [2,"c"]]

within pair be enumerate(["red","green","blue"]) ::
  speak(pair[0] + ": " + pair[1])
;;
// 0: red
// 1: green
// 2: blue
```

`zip` — Pair Two Lists

Combine two lists element-by-element:

```
forge keys be ["name", "age", "city"]
forge vals be ["Alice", 30, "NYC"]
speak(zip(keys, vals))
// [["name","Alice"], ["age",30], ["city","NYC"]]
```

Type System & Conversion

Getting Type — `essence()`

Returns the type name as a string:

```
speak(essence(42))          // "number"
speak(essence("hi"))        // "text"
speak(essence(yep))         // "truth"
speak(essence(void))        // "void"
speak(essence([1,2,3]))     // "list"
speak(essence(::;;))        // "tome"
```

Converting Types — `morph()`

```
speak(morph("42", "number"))     // 42
speak(morph(42, "text"))         // "42"
speak(morph(1, "truth"))         // yep
speak(morph(0, "truth"))         // nope
speak(morph("3.14", "number"))   // 3.14
```

Type Checking Helpers

```
speak(essence(x) equals "number")      // Check if number
speak(essence(x) equals "text")        // Check if string
speak(essence(x) equals "list")        // Check if list
speak(essence(x) equals "tome")        // Check if dict
speak(x equals void)                   // Check if null
```

Collections

Set

A collection of unique elements. Duplicate additions are silently ignored.

```
forge s be Set()
s.add(1)
s.add(2)
s.add(2)    // Duplicate ignored
s.add(3)

speak(s.size())     // 3
speak(s.has(2))     // yep
speak(s.has(99))    // nope
speak(s.values())   // [1, 2, 3]

s.remove(2)
speak(s.values())   // [1, 3]

s.clear()
speak(s.size())     // 0
speak(s.isEmpty())  // yep
```

Set methods: `add(v)`, `remove(v)`, `has(v)`, `values()`, `size()`, `isEmpty()`, `clear()`, `toList()`

Map

A key-value store where keys can be any type (including objects):

```
forge m be Map()
m.set("name", "Alice")
m.set("age", 30)
m.set("active", yep)

speak(m.get("name"))     // Alice
speak(m.has("age"))      // yep
```

```
speak(m.keys())          // ["name", "age", "active"]
speak(m.values())        // ["Alice", 30, yep]
speak(m.entries())       // [["name","Alice"], ["age",30], ["active
speak(m.size())          // 3

m.delete("active")
speak(m.size())          // 2
m.clear()
```

Map methods: `set(k, v)`, `get(k)`, `has(k)`, `delete(k)`, `keys()`, `values()`, `entries()`, `size()`, `isEmpty()`, `clear()`

Queue (FIFO)

First-in, first-out data structure:

```
forge q be Queue()
q.enqueue("first")
q.enqueue("second")
q.enqueue("third")

speak(q.peek())      // first  (look without removing)
speak(q.dequeue())   // first  (remove and return)
speak(q.peek())      // second
speak(q.size())      // 2
speak(q.isEmpty())   // nope
```

Queue methods: `enqueue(v)`, `dequeue()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

Stack (LIFO)

Last-in, first-out data structure:

```
forge s be Stack()
s.push(10)
```

```
s.push(20)
s.push(30)

speak(s.peek())   // 30  (look without removing)
speak(s.pop())    // 30  (remove and return)
speak(s.pop())    // 20
speak(s.size())   // 1

// Stack-based undo system
forge history be Stack()
history.push("action1")
history.push("action2")
history.push("action3")

speak("Undoing:", history.pop())  // action3
speak("Undoing:", history.pop())  // action2
```

Stack methods: `push(v)`, `pop()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

LinkedList

A doubly-linked list with O(1) front/back operations:

```
forge list be LinkedList()
list.append(1)
list.append(2)
list.append(3)
list.prepend(0)

speak(list.toList())   // [0, 1, 2, 3]
speak(list.size())     // 4
speak(list.get(2))     // 2

list.remove(1)         // Remove element with value 1
speak(list.toList())   // [0, 2, 3]

speak(list.head())     // 0
```

```
speak(list.tail())     // 3
```

LinkedList methods: `append(v)`, `prepend(v)`, `remove(v)`, `get(idx)`, `size()`, `head()`, `tail()`, `toList()`, `clear()`

Matrix Operations

sdev has a comprehensive built-in matrix math library useful for data science and machine learning.

Creating Matrices

```
// Matrix filled with a value
forge zeros be matrix(3, 3, 0)
// [[0,0,0], [0,0,0], [0,0,0]]

forge ones be matrix(2, 4, 1)
// [[1,1,1,1], [1,1,1,1]]

// Identity matrix
forge eye be identity(4)
// [[1,0,0,0], [0,1,0,0], [0,0,1,0], [0,0,0,1]]

// Reshape a flat list into a matrix
forge m be reshape([1,2,3,4,5,6], 2, 3)
// [[1,2,3], [4,5,6]]
```

Matrix Arithmetic

```
forge a be [[1, 2], [3, 4]]
forge b be [[5, 6], [7, 8]]

// Element-wise operations
```

```
speak(matadd(a, b))     // [[6,8], [10,12]]
speak(matsub(a, b))     // [[-4,-4], [-4,-4]]
speak(matscale(a, 2))   // [[2,4], [6,8]]

// Matrix multiplication
speak(matmul(a, b))
// [[1*5+2*7, 1*6+2*8], [3*5+4*7, 3*6+4*8]]
// = [[19,22], [43,50]]

// Transpose
speak(transpose(a))
// [[1,3], [2,4]]
```

Vector Operations

```
forge v1 be [1, 2, 3]
forge v2 be [4, 5, 6]

// Dot product
speak(dot(v1, v2))   // 1*4 + 2*5 + 3*6 = 32

// Element-wise multiply via each
forge elem be each(v1, (x, i) -&gt; x * v2[i])
speak(elem)   // [4, 10, 18]
```

Matrix Utilities

| Function | Description | |----------|----------| | `matrix(rows, cols, fill)` | Create filled matrix | | `identity(n)` | Create n×n identity matrix | | `transpose(m)` | Transpose rows and columns | | `matmul(a, b)` | Matrix multiplication | | `matadd(a, b)` | Element-wise addition | | `matsub(a, b)` | Element-wise subtraction | | `matscale(m, s)` | Scalar multiplication | | `dot(a, b)` | Dot product of

two vectors | | `reshape(list, rows, cols)` | Reshape list to 2D matrix | | `flatten(m)` | Flatten 2D matrix to 1D list | | `shape(m)` | Get `[rows, cols]` of matrix | | `sum(m)` | Sum all elements | | `mean(list)` | Mean of a 1D list |

Neural Network Example

```
// Sigmoid activation function
conjure sigmoid(x) ::
  yield 1 / (1 + exp(-x))
;;

// Feedforward neural network layer
conjure layer(inputs, weights, biases) ::
  // inputs: [1 x n], weights: [n x m], biases: [1 x m]
  forge z be matmul(inputs, weights)
  forge zb be matadd(z, biases)
  // Apply sigmoid element-wise
  yield each(zb, row -&gt; each(row, x -&gt; sigmoid(x)))
;;

forge input be [[0.5, 0.3, 0.2]]
forge w1 be [[0.1, 0.2], [0.3, 0.4], [0.5, 0.6]]
forge b1 be [[0.1, 0.1]]

forge output be layer(input, w1, b1)
speak("Layer output:", output)
```

Graphics & Game Development

sdev has a built-in 2D graphics and turtle graphics API. Graphics commands render to the Canvas panel in the IDE.

Canvas Setup

```
canvas(800, 600)    // Set canvas dimensions (width, height)
clear("#1a1a2e")    // Clear with a background color
```

Fill and Stroke

```
fill("blue")          // Set fill color
noFill()              // Disable fill
stroke("red", 2)      // Set stroke: color, width
noStroke()            // Disable stroke
lineWidth(3)          // Set line width separately
alpha(0.5)            // Set global transparency (0–1)
lineCap("round")      // Line cap: "round", "square", "butt"
lineJoin("round")     // Line join: "round", "bevel", "miter"
```

Basic Shapes

```
// Rectangle: rect(x, y, width, height, cornerRadius?)
fill("steelblue")
rect(50, 50, 200, 100)
rect(300, 50, 200, 100, 20)    // Rounded corners

// Circle: circle(x, y, radius)
fill("tomato")
circle(150, 300, 60)

// Ellipse: ellipse(x, y, radiusX, radiusY, rotation?)
fill("gold")
ellipse(350, 300, 100, 50)
```

```
ellipse(550, 300, 100, 50, 45)   // Rotated 45°

// Line: line(x1, y1, x2, y2)
stroke("white", 2)
line(50, 400, 750, 400)

// Arc: arc(x, y, radius, startAngle, endAngle, counterclockwise?)
stroke("lime", 3)
arc(200, 500, 50, 0, 270)

// Triangle: triangle(x1, y1, x2, y2, x3, y3)
fill("orchid")
triangle(400, 450, 450, 550, 350, 550)

// Polygon: polygon([[x,y], ...])
fill("coral")
polygon([[600, 400], [650, 430], [640, 490], [560, 490], [550, 430

// Star: star(x, y, outerR, innerR, points?)
fill("gold")
star(150, 150, 50, 25)          // 5-pointed star
star(300, 150, 60, 25, 8)       // 8-pointed star

// Heart: heart(x, y, size)
fill("red")
heart(500, 150, 50)

// Point: point(x, y, size?)
fill("white")
point(100, 100)
point(150, 100, 8)
```

Text Drawing

```
fill("white")
text("Hello, World!", 100, 100)           // Basic text
text("Large Text", 100, 150, 36)          // With size
text("Bold Text", 100, 200, 24, "bold")   // With style
```

```
font("Arial", "bold")                    // Set font family and s
textAlign("center", "middle")            // Horizontal: left/cente
```

Gradients

```
// Linear gradient: fill with gradient from (x1,y1) to (x2,y2)
// Color stops: [position, color] where position is 0.0 to 1.0
linearGradient(0, 0, 400, 0, [0, "red"], [0.5, "yellow"], [1, "gree
rect(0, 0, 400, 100)

// Radial gradient: from inner circle to outer circle
radialGradient(200, 200, 0, 200, 200, 150, [0, "white"], [0.5, "cya
circle(200, 200, 150)
```

Shadows

```
shadow("rgba(0,0,0,0.5)", 15, 5, 5)   // color, blur, offsetX, offs
fill("gold")
circle(200, 200, 80)
noShadow()
```

Transformations

```
save()                // Push current state
translate(200, 200)   // Move origin to (200, 200)
rotate(0.785)         // Rotate 45° (in radians)
scale(1.5)            // Scale uniformly
rect(-50, -50, 100, 100)   // Draw centered rectangle
restore()             // Pop saved state

resetTransform()      // Reset all transforms to identity
```

Path Drawing

```
// Custom polygon path
fill("purple")
beginPath()
moveTo(100, 100)
lineTo(200, 80)
lineTo(250, 150)
lineTo(200, 220)
lineTo(100, 200)
closePath()
fillPath()
strokePath()

// Bezier curve
stroke("cyan", 2)
beginPath()
moveTo(100, 300)
bezierTo(150, 200, 250, 400, 300, 300)   // cp1x, cp1y, cp2x, cp2y
strokePath()

// Quadratic curve
beginPath()
moveTo(100, 400)
quadraticTo(200, 350, 300, 400)           // cpx, cpy, x, y
strokePath()
```

Turtle Graphics

The turtle graphics API lets you draw by commanding a turtle that moves and draws lines as it goes.

```
canvas(500, 500)
clear("#0d0d20")
turtle()           // Create turtle at center, facing up

// Basic movement
forward(100)       // Move forward 100 units (drawing as it goes)
right(90)          // Turn right 90 degrees
forward(100)
```

```
right(90)
forward(100)
right(90)
forward(100)       // This draws a square!

// Pen control
penup()            // Lift pen — move without drawing
goto(250, 250)     // Jump to coordinates
pendown()          // Lower pen — start drawing again

// Pen style
pencolor("lime")   // Set pen color (any CSS color)
penwidth(3)        // Set pen width

// Advanced
setheading(45)     // Set absolute direction (degrees)
speak(heading())   // Get current heading
speak(pos())       // Get current [x, y] position
home()             // Return to start position

// Turtle circle
turtleCircle(50)        // Draw full circle
turtleCircle(50, 72)    // Polygon approximation (72 steps)

// Dot and stamp
dot(10, "red")     // Draw a filled dot
stamp()            // Leave a mark of the turtle shape

// EXAMPLE: Psychedelic spiral
canvas(600, 600)
clear("#000000")
turtle()
within i be sequence(0, 300) ::
  pencolor(hue(i * 1.2))
  penwidth(1 + i * 0.02)
  forward(i)
  right(91)
;;
```

Sprites & Game Objects

```
// Create sprite: createSprite(x, y, width, height, color?)
forge player be createSprite(100, 100, 40, 40, "blue")
forge enemy be createSprite(400, 300, 40, 40, "red")

// Access/set properties
player.x be 150
player.velocityX be 5
player.velocityY be -2

// Update position (applies velocity)
updateSprite(player)

// Draw sprite
drawSprite(player)
drawSprite(enemy)

// Collision detection
ponder spriteCollides(player, enemy) ::
  speak("HIT!")
;;

// Move by delta
moveSprite(player, 10, 5)
```

Color Functions

```
// CSS color names work directly
fill("red")
fill("steelblue")
fill("rgba(255, 0, 0, 0.5)")

// Helper functions
forge red be rgb(255, 0, 0)
forge transparent be rgba(0, 255, 0, 0.3)

// HSL color — hue(hue, saturation?, lightness?)
```

```
forge cyan be hue(180)              // Cyan (s=100, l=50 defaults)
forge purple be hue(270, 80, 60)    // Custom HSL
forge faded be hsla(240, 100, 50, 0.4)

// Rainbow cycle (great for spirals!)
within i be sequence(360) ::
  fill(hue(i))
  rect(i, 0, 1, 50)
;;

// Random color
fill(randomColor())
```

Graphics Math

```
// Convert degrees/radians
speak(radians(180))   // 3.14159...
speak(degrees(PI))    // 180

// Lerp (smooth transitions)
forge pos be lerp(0, 100, 0.25)   // 25

// Map value between ranges
forge brightness be mapRange(50, 0, 100, 0, 255)   // 127.5

// Clamp
forge safe be clamp(150, 0, 100)   // 100

// Distance between two points
speak(dist(0, 0, 3, 4))   // 5
```

Complete 2D Graphics Example — Starfield

```
canvas(800, 600)
```

```
// Create 200 stars
forge stars be each(sequence(200), i -&gt; ::
  "x": randint(0, 800),
  "y": randint(0, 600),
  "r": chaos() * 2 + 0.5,
  "speed": chaos() * 2 + 0.5,
  "bright": chaos() * 0.7 + 0.3
;;)

clear("#000011")

within star be stars ::
  fill(rgba(255, 255, 255, star.bright))
  circle(star.x, star.y, star.r)
;;

// Draw nebula in center
radialGradient(400, 300, 0, 400, 300, 200,
  [0, "rgba(100,50,200,0.4)"],
  [0.5, "rgba(50,100,255,0.2)"],
  [1, "rgba(0,0,0,0)"]
)
circle(400, 300, 200)
```

File I/O

> ***Note:*** *File I/O is available in the Python standalone interpreter and native desktop builds. In the browser-based IDE, these functions are simulated.*

Reading Files

```
// Read entire file as text
```

```
forge content be decipher("data.txt")
speak(content)

// Read and parse JSON
forge config be unetch(decipher("config.json"))
speak(config.version)
speak(config.name)
```

Writing Files

```
// Write text file
inscribe("output.txt", "Hello, World!\n")

// Write JSON with indentation
forge data be :: "score": 100, "name": "Alice", "level": 5 ;;
inscribe("save.json", etch(data))
```

Append to File

```
appendFile("log.txt", "User logged in\n")
appendFile("log.txt", "User viewed dashboard\n")
```

File Utilities

Function	Description
`decipher(path)`	Read file as text
`inscribe(path, content)`	Write text to file
`appendFile(path, content)`	Append to file
`fileExists(path)`	Returns `yep`/`nope`
`deleteFile(path)`	Delete a file
`listDir(path)`	Get list of files in directory

Networking

> ***Note:*** *Network functions are available in async contexts. In the browser they use* `fetch`*. In native builds they use Node.js http.*

HTTP GET

```
async conjure getUsers() ::
  forge data be await fetch("https://jsonplaceholder.typicode.com/u
  yield data
;;
```

HTTP POST

```
async conjure createUser(name, email) ::
  forge response be await fetch("https://api.example.com/users", :
    "method": "POST",
    "headers": :: "Content-Type": "application/json" ;;,
    "body": :: "name": name, "email": email ;;
  ;;)
  yield response
;;
```

Examples & Recipes

FizzBuzz

```
within i be sequence(1, 101) ::
  ponder i % 15 equals 0 ::
```

```
    speak("FizzBuzz")
  ;; otherwise ponder i % 3 equals 0 ::
    speak("Fizz")
  ;; otherwise ponder i % 5 equals 0 ::
    speak("Buzz")
  ;; otherwise ::
    speak(i)
  ;;
;;
```

Fibonacci (Memoized)

```
forge memo be :: ;;

conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  ponder contains(memo, morph(n, "text")) ::
    yield memo[morph(n, "text")]
  ;;
  forge result be fib(n - 1) + fib(n - 2)
  memo[morph(n, "text")] be result
  yield result
;;

within i be sequence(0, 30) ::
  speak("fib(" + morph(i, "text") + ") =", fib(i))
;;
```

Binary Search

```
conjure binarySearch(arr, target) ::
  forge lo be 0
  forge hi be measure(arr) - 1

  cycle lo &lt;= hi ::
    forge mid be ground((lo + hi) / 2)
    ponder arr[mid] equals target ::
```

```
      yield mid
    ;; otherwise ponder arr[mid] &lt; target ::
      lo be mid + 1
    ;; otherwise ::
      hi be mid - 1
    ;;
  ;;

  yield -1
;;

forge sorted be [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
speak(binarySearch(sorted, 23))   // 5
speak(binarySearch(sorted, 50))   // -1
```

Quicksort

```
conjure quicksort(arr) ::
  ponder measure(arr) &lt;= 1 :: yield arr ;;

  forge pivot be arr[ground(measure(arr) / 2)]
  forge less be sift(arr, x -&gt; x &lt; pivot)
  forge equal be sift(arr, x -&gt; x equals pivot)
  forge greater be sift(arr, x -&gt; x &gt; pivot)

  yield quicksort(less) + equal + quicksort(greater)
;;

speak(quicksort([3, 6, 8, 10, 1, 2, 1]))
// [1, 1, 2, 3, 6, 8, 10]
```

Stack-Based Calculator

```
conjure calculate(expression) ::
  forge tokens be shatter(expression, " ")
  forge stack be Stack()
```

```
  within token be tokens ::
    ponder token equals "+" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a + b)
    ;; otherwise ponder token equals "-" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a - b)
    ;; otherwise ponder token equals "*" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a * b)
    ;; otherwise ponder token equals "/" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a / b)
    ;; otherwise ::
      stack.push(morph(token, "number"))
    ;;
  ;;

  yield stack.pop()
;;

speak(calculate("3 4 + 2 *"))    // (3+4)*2 = 14
speak(calculate("5 1 2 + 4 * + 3 -"))  // 5+(1+2)*4-3 = 14
```

Caesar Cipher

```
conjure caesarEncode(text, shift) ::
  forge result be ""
  within ch be chars(text) ::
    forge code be locate("ABCDEFGHIJKLMNOPQRSTUVWXYZ", upper(ch))
    ponder code &gt;= 0 ::
      forge shifted be (code + shift) % 26
      result be result + "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[shifted]
    ;; otherwise ::
```

```
      result be result + ch
    ;;
  ;;
  yield result
;;

speak(caesarEncode("HELLO WORLD", 3))   // KHOOR ZRUOG
speak(caesarEncode("KHOOR ZRUOG", 23))  // HELLO WORLD (decode with
```

Word Frequency Counter

```
conjure wordFrequency(text) ::
  forge words be shatter(lower(trim(text)), " ")
  forge freq be ::;;

  within word be words ::
    forge w be replace(replace(word, ".", ""), ",", "")
    ponder measure(w) &gt; 0 ::
      ponder contains(freq, w) ::
        freq[w] be freq[w] + 1
      ;; otherwise ::
        freq[w] be 1
      ;;
    ;;
  ;;

  yield freq
;;

forge text be "the quick brown fox jumps over the lazy dog the fox
forge freq be wordFrequency(text)
speak(freq)
// Sort by frequency
forge sorted be sort(inscriptions(freq), k -&gt; -freq[k])
```

Prime Sieve

```
conjure sieve(limit) ::
  forge primes be each(sequence(2, limit + 1), x -&gt; x)
  forge result be []

  cycle measure(primes) &gt; 0 ::
    forge p be primes[0]
    gather(result, p)
    primes be sift(primes, x -&gt; x % p differs 0)
  ;;

  yield result
;;

speak(sieve(50))
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Linked-List Graph Traversal (BFS)

```
// BFS using a Queue
conjure bfs(graph, start) ::
  forge visited be Set()
  forge queue be Queue()
  forge order be []

  queue.enqueue(start)
  visited.add(start)

  cycle isnt queue.isEmpty() ::
    forge node be queue.dequeue()
    gather(order, node)

    within neighbor be graph[node] ::
      ponder isnt visited.has(neighbor) ::
        visited.add(neighbor)
        queue.enqueue(neighbor)
      ;;
    ;;
  ;;
```

```
  yield order
;;

forge graph be ::
  "A": ["B", "C"],
  "B": ["A", "D", "E"],
  "C": ["A", "F"],
  "D": ["B"],
  "E": ["B", "F"],
  "F": ["C", "E"]
;;

speak(bfs(graph, "A"))  // ["A", "B", "C", "D", "E", "F"]
```

Turtle Mandala

```
canvas(600, 600)
clear("#000000")
turtle()
penwidth(1.5)

forge petals be 12
forge petalAngle be 360 / petals

within i be sequence(petals) ::
  pencolor(hue(i * petalAngle))
  within j be sequence(60) ::
    forward(j * 0.5)
    right(6)
  ;;
  penup()
  home()
  pendown()
  setheading(i * petalAngle)
;;
```

JavaScript Interop (JS Interpreter Only)

The browser-based interpreter includes a special `js` keyword for calling JavaScript functions directly from sdev code. This allows integration with browser APIs, DOM manipulation, and third-party JS libraries.

Three Syntax Forms

```
// 1. Single-line expression
js &lt;expression&gt;

// 2. Parenthesized form (for objects, multi-line expressions)
js (
  &lt;expression&gt;
)

// 3. Statement block
js {
  &lt;statements&gt;
}
```

Basic Examples

```
// Browser dialog
js alert("Hello from sdev!")

// Read browser properties
forge width be js window.innerWidth
speak("Width:", width)

// DOM manipulation
js document.body.style.backgroundColor = "#1a1a2e"
```

```
js document.title = "My sdev App"

// Math
forge angle be js Math.PI / 6
forge result be js Math.sin(0.5)
```

Object Literals

```
// Use parenthesized form for object literals
forge opts be js ({
  dragging: true,
  zoom: 12,
  center: [51.505, -0.09]
})

// Nested objects
forge config be js ({
  api: {
    url: "https://api.example.com",
    timeout: 5000
  },
  debug: false
})
```

Arrow Functions

```
// Map/filter in JS
forge doubled be js [1, 2, 3, 4].map(x =&gt; x * 2)
forge filtered be js [1, 2, 3, 4].filter(x =&gt; x &gt; 2)

// Event handlers
js document.getElementById("btn").addEventListener("click", (e) =&g
  console.log("Clicked!", e.target)
})

// Use sdev variables in JS arrows
forge multiplier be 5
```

```
forge result be js [1, 2, 3].map(x =&gt; x * multiplier)
```

Multi-line JS Blocks

```
js {
  const canvas = document.createElement("canvas")
  canvas.width = 400
  canvas.height = 300
  document.body.appendChild(canvas)

  const ctx = canvas.getContext("2d")
  ctx.fillStyle = "navy"
  ctx.fillRect(0, 0, 400, 300)
  ctx.fillStyle = "white"
  ctx.font = "24px monospace"
  ctx.fillText("Made in sdev!", 100, 150)
}
```

Leaflet Map Integration

```
// Initialize a Leaflet map
forge map be js L.map("map-container").setView([51.505, -0.09], 13

js L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
  attribution: "© OpenStreetMap contributors"
})).addTo(map)

// Add markers
conjure addMarker(lat, lng, label) ::
  js L.marker([lat, lng]).addTo(map).bindPopup(label)
;;

addMarker(51.505, -0.09, "London, UK")
addMarker(48.8566, 2.3522, "Paris, France")
addMarker(52.52, 13.405, "Berlin, Germany")
```

Complete Reference Card

Keywords

| Keyword | Purpose | |---------|---------| | `forge` | Declare a variable | | `be` | Assign a value | | `conjure` | Declare a function | | `yield` | Return a value from a function | | `ponder` | `if` statement | | `otherwise` | `else` clause | | `cycle` | `while` loop | | `iterate` | `for-each` loop (iterate x through list) | | `through` | Used with iterate | | `within` | `for-in` loop (within x be list) | | `yeet` | `break` — exit loop | | `skip` | `continue` — skip iteration | | `attempt` | `try` block | | `rescue` | `catch` block | | `yep` | Boolean `true` | | `nope` | Boolean `false` | | `void` | Null / absent value | | `also` | Logical AND | | `either` | Logical OR | | `isnt` | Logical NOT | | `equals` | Equality comparison | | `differs` | Inequality comparison | | `essence` | Define a class | | `extend` | Class inheritance | | `self` | Instance reference inside class methods | | `super` | Parent class reference | | `new` | Create class instance | | `summon` | Import from Gist | | `async` | Mark function as async | | `await` | Wait for async operation | | `spawn` | Run function concurrently |

Special Symbols

| Symbol | Purpose | |--------|---------| | `::` | Start a block | | `;;` | End a block | | `->` | Lambda arrow | | `\|>` | Pipe operator | | `~` | Ternary condition operator | | `:` | Ternary else / dict key-value separator | | `//` | Line comment | | `#`

| Line comment (Python style) | | `^` | Power operator | | `<>` | Inequality (alternative to `differs`) |

Built-in Constants

Constant	Value
`PI`	3.141592653589793
`TAU`	6.283185307179586
`E`	2.718281828459045
`INFINITY`	Infinity

Quick Reference Examples

```
// ── Variables ──
forge x be 42
forge name be "Alice"
forge active be yep

// ── Arithmetic ──
speak(2 ^ 10)     // 1024
speak(10 % 3)     // 1
speak(-5 + 8)     // 3

// ── String ──
speak("Hello" + " " + "World")
speak(upper("hello"))        // HELLO
speak(measure("hello"))      // 5

// ── List ──
forge nums be [1, 2, 3, 4, 5]
gather(nums, 6)              // [1,2,3,4,5,6]
speak(nums[-1])              // 6
speak(sort(nums))            // [1,2,3,4,5,6]

// ── Conditionals ──
ponder x &gt; 0 ::
  speak("positive")
;; otherwise ::
```

```
  speak("non-positive")
;;

// ── Ternary ──
forge label be x &gt; 0 ~ "pos" : "neg"

// ── Loop ──
within i be sequence(5) :: speak(i) ;;  // 0 1 2 3 4
cycle x &gt; 0 :: x be x - 1 ;;

// ── Function ──
conjure add(a, b) :: yield a + b ;;
forge double be x -&gt; x * 2

// ── Pipe ──
[1,2,3,4,5] |&gt; each(x -&gt; x * 2) |&gt; sift(x -&gt; x &gt; 4)
// [6, 8, 10]

// ── OOP ──
essence Dog ::
  conjure init(self, name) :: self.name be name ;;
  conjure bark(self) :: speak(self.name + ": WOOF!") ;;
;;
forge rex be new Dog("Rex")
rex.bark()

// ── Error Handling ──
attempt ::
  forge n be morph("not a number", "number")
;; rescue err ::
  speak("Error:", err)
;;

// ── Higher-order ──
forge evens be sift(sequence(10), x -&gt; x % 2 equals 0)
forge sum be fold(evens, 0, (a, b) -&gt; a + b)
speak(sum)   // 20

// ── Collections ──
```

```
forge s be Set()
s.add(1) s.add(2) s.add(2)
speak(s.values())   // [1, 2]

forge q be Queue()
q.enqueue("a") q.enqueue("b")
speak(q.dequeue())   // a

// ── Matrix ──
forge a be [[1,2],[3,4]]
forge b be [[5,6],[7,8]]
speak(matmul(a, b))   // [[19,22],[43,50]]

// ── Graphics ──
canvas(400, 400)
clear("#111")
fill(hue(120))
circle(200, 200, 100)

// ── Turtle ──
turtle()
within i be sequence(4) ::
  forward(100)
  right(90)
;;
```

This documentation covers sdev version 1.x. For the latest updates and additional examples, visit the sdev IDE.

ГЛАВА 07

Част II — UI инструментариум

Част II — UI инструментариум

Тази част документира всеки UI примитив, добавен след първото издание. Инструментариумът е общ за JavaScript и Python интерпретаторите; програма, която използва само документираното API, ще работи без промяна и в двата.

Глава U1 — Прозорци в дълбочина

Прозорец е контейнерът от най-високо ниво на всяка визуална програма. Всеки компонент живее в някой прозорец. Отваряте прозорец с `window(заглавие, ширина, височина)` и го затваряте с `endwindow`.

```
window("Здравей", 480, 320)
  heading("Здравей, свят!", 1)
  paragraph("Това е първият ми прозорец.")
endwindow
```

Размери

Двата числови аргумента са **началните** ширина и височина в CSS пиксели. И двата интерпретатора ги

третират като *препоръка*: браузърът ги уважава, но позволява на потребителя да преоразмери прозореца с мишката, а Python интерпретаторът ги прилага като начални вътрешни размери на Tk прозорец.

Често използвани размери:

| За какво | Препоръка | |----------|----------| | Изскачащ диалог | `320 × 200` | | Форма / настройки | `480 × 600` | | Панел редактор | `720 × 540` | | Табло | `960 × 640` | | Цял екран | `1280 × 800` | | Малка помощна | `280 × 160` | | Мобилен формат | `360 × 640` |

Няколко прозореца

Можете да отворите повече от един прозорец в една програма. Всеки `window(...)` блок е независим; компонентите в прозорец Б не виждат тези в прозорец А, освен ако не споделят име на `uiget`/`uiset`.

Затваряне на прозорец от код

Повикайте `closewindow(заглавие)`, за да затворите прозорец. За да го отворите отново, просто изпълнете блока — ако вече е отворен, се игнорира.

Персонализация на прозорец

След размера може да подадете **тома с опции**:

```
window("Моето приложение", 480, 600, {
  resizable: true,
  alwaysOnTop: false,
  background: "#0F172A",
  foreground: "#F8FAFC",
  padding: 16,
  radius: 12,
  centered: true
})
```

Всички ключове са опционални; интерпретаторът тихо игнорира непознати ключове.

Глава U2 — Бутони в дълбочина

`button(етикет, onclick)` е най-използваният компонент. Подразбиращата се форма е цветен бутон, който извиква функция при клик.

```
button("Натисни ме", () -&gt; speak("кликнато!"))
```

Варианти

Третият аргумент избира визуален вариант:

| Вариант | Изглежда като | За какво | |---------|---------------|----------| | `primary` | Плътен акцентен цвят, бял текст | Главно действие на екрана. | | `secondary` | Мек фон, тъмен текст | Спомагателни действия. | | `ghost` | Прозрачен до hover | Третични

действия, тулбари. | | `outline` | Само рамка | Филтри, сегментни контроли. | | `destructive` | Червен фон | Изтриване или необратимо действие. | | `link` | Изглежда като линк | Навигация в текст. |

Тома за персонализация

Четвъртият аргумент позволява да се промени почти всичко:

```
button("Запис", saveFn, "primary", {
  width: 160,
  height: 44,
  radius: 10,
  background: "#0891B2",
  foreground: "#FFFFFF",
  hoverBackground: "#0E7490",
  shadow: "0 4px 12px rgba(0,0,0,.25)",
  icon: "save",
  url: "https://example.com",
  newTab: true,
  disabled: false,
  tooltip: "Запазва документа"
})
```

Ако е подаден `url`, бутонът се държи и като линк — клик го отваря (в нов таб при `newTab: true`) **след** изпълнение на `onclick`.

Иконки

Подайте `icon: "<име>"`, за да добавите Lucide иконка вляво от етикета.

```
row()
  button("", deleteRow, "ghost", {icon: "trash",  tooltip: "Изтрий
  button("", editRow,   "ghost", {icon: "edit",   tooltip: "Редакт
endrow
```

Деактивиране

Задайте `disabled: true`, за да направите бутона сив. Комбинирайте със състояние, за да заключите UI по време на операция:

```
button("Изпрати", onSubmit, "primary", {disabled: uiget("loading")
```

Глава U3 — Текстови полета и форми

Всеки входен компонент приема `име` като първи аргумент. Стойността е достъпна навсякъде чрез `uiget(име)`, а `uiset(име, стойност)` я променя.

`input` — еднородово

```
input("email", "you@example.com")
button("Поздрави", () -&gt; speak("Здравей " + uiget("email")))
```

Персонализация:

```
input("email", "you@example.com", "", {
  width: 320,
  type: "email",
  maxLength: 80,
```

```
    required: true,
    autocomplete: "email",
    background: "#0F172A",
    foreground: "#E2E8F0",
    border: "#334155",
    radius: 8,
    padding: 10
  })
```

textarea — многоредово

```
textarea("note", 6, "Напишете нещо...")
```

Четвъртият аргумент поддържа `width`, `height`, `resizable`, `monospace`, `wrap`.

checkbox

```
checkbox("agree", "Съгласен съм с условията")
if uiget("agree")
  speak("Благодаря")
end
```

Вторият аргумент е видимият етикет. За checkbox без етикет (например в клетка на таблица) подайте празен низ.

slider и select

```
slider("volume", 0, 100, 5)
select("theme", ["light", "dark", "auto"])
```

Глава U4 — Оформления

По подразбиране компонентите се подреждат вертикално, отгоре надолу. За да промените това, обвийте ги в блок за оформление.

Редове и колони

```
row()
  button("Отказ", cancelFn, "ghost")
  button("Запис", saveFn,   "primary")
endrow
```

`row()` приема тома: `gap`, `align`, `justify`, `wrap`.

Групи

Блок `group(заглавие)` рисува рамка около децата — идеално за свързани полета на форма.

```
group("Профил")
  input("name", "Вашето име")
  input("email", "you@example.com")
endgroup
```

Табове

```
tabs()
  tab("Общи")
    input("name", "Име на проекта")
  endtab
  tab("Разширени")
    checkbox("experimental", "Включи експериментални функции")
```

```
  endtab
endtabs
```

Таблици

```
table(
  ["Име", "Възраст", "Държава"],
  [
    ["Ада",  36, "UK"],
    ["Линус", 54, "FI"]
  ]
)
```

Глава U5 — Реактивно състояние с `uiget` / `uiset`

Всеки компонент с `име` излага текущата си стойност чрез `uiget(име)`. Стойността е реактивна: всеки израз в същия прозорец, който я чете, ще се преизчисли при промяна.

```
input("first", "Ада")
input("last",  "Лъвлейс")
paragraph("Здравей, " + uiget("first") + " " + uiget("last") + "!"
```

За програмна промяна — `uiset(име, стойност)`. Така се правят бутоните за нулиране, изчислимите полета и зареждането на данни от мрежата:

```
button("Нулирай", () -&gt; uiset("first", "") and uiset("last", ""
```

Глава U6 — Справочник за персонализация

Тази глава е една голяма таблица за справка. Всеки ред е ключ, който може да подадете в томата с опции на всеки компонент.

Ключ	Тип	Прилага се към	Бележки
`width`	число (px)	всички	Фиксирана ширина.
`height`	число (px)	всички	Фиксирана височина.
`padding`	число или [t,r,b,l]	контейнери, бутони, полета	Вътрешни отстъпи.
`margin`	число или [t,r,b,l]	всички	Външни отстъпи.
`radius`	число (px)	повечето	Радиус на ъглите.
`background`	цвят	повечето	Плътен фон.
`foreground`	цвят	текстови	Цвят на текста.
`border`	цвят	повечето	Цвят на рамката.
`borderWidth`	число (px)	повечето	Дебелина на рамката.
`shadow`	CSS shadow низ	прозорец, бутон, група	Сянка.
`font`	низ	текстови	CSS font-family.
`fontSize`	число (px)	текстови	Размер на шрифта.
`fontWeight`	число	текстови	Удебеленост.
`align`	start|center|end|stretch	row, column, group	Cross-axis.
`justify`	start|center|end|between|around	row, column	Main-axis.
`gap`	число (px)	row, column, group	Разстояние между децата.
`wrap`	true|false	row	Позволява пренасяне.
`icon`	име на иконка	button, menuitem	Lucide иконка.
`tooltip`	низ	повечето	Помощен текст при hover.
`disabled`	boolean	полета, бутони	Сив и без

събития. | | `hidden` | boolean | всички | Премахва от оформлението. | | `url` | низ | button, label | Клик отваря URL. | | `newTab` | boolean | button, label | Отваря URL в нов таб. | | `onChange` | функция | полета | При всяка промяна. | | `onFocus/onBlur` | функция | полета | | | `hoverBackground` | цвят | button | | | `activeBackground` | цвят | button | | | `focusRing` | цвят | полета, бутони | | | `placeholderColor` | цвят | полета, textarea | | | `rows/cols` | число | textarea | Начален размер. | | `resizable` | true|false | window, textarea | | | `monospace` | boolean | textarea, label | Моноширинен шрифт. | | `wrap` | "hard"|"soft" | textarea | Политика на пренасяне. |

Глава U7 — Облачно работно пространство и автоматична синхронизация

Започвайки от това издание, IDE пази цялото ви работно пространство — файлове **и** дървото от вложени папки — в облака. Не е нужно да мислите за това.

Как работи автоматичната синхронизация

Всяка промяна в редактора се отлага (≈ 800 мс) и се записва в облака като малък patch. При презареждане IDE взима последната снимка преди

да рисува нещо, така че виждате последно записаното.

Ако два таба редактират един файл едновременно, **последният запис печели** на ниво файл, но промените на „загубилия“ остават в съседен файл `<оригинал>.conflict-<timestamp>`. Отворете го, копирайте каквото ви трябва и го изтрийте.

Ръчно записване

От меню **File → Save to cloud** (Ctrl + S) изпразва незаписаните промени веднага. Полезно преди публикуване или споделяне.

Офлайн режим

Ако мрежата изчезне, IDE продължава да работи срещу копие в паметта. Малък банер в статус бара ви предупреждава. При връщане на връзката всички чакащи промени се изпращат по ред.

Вложени папки

Дървото с файлове е истинско дърво. Десен клик върху папка — **New file**, **New folder**, **Rename**, **Delete**, **Duplicate**, **Move...**. Drag-and-drop работи между всеки две папки.

Програмите достъпват работното пространство по време на изпълнение чрез `readFile(път)` и

`writeFile(път, съдържание)`. Пътищата са POSIX-стил с корен в работното пространство.

ГЛАВА 08

Справочни карти за компоненти

Справочни карти за компоненти

По една карта на компонент, по азбучен ред.

`audio`

Signature. `audio(src)`

Description. Аудио плейър.

Example.

```
audio(src)
```

`button`

Signature. `button(етикет, onclick, вариант?)`

Description. Кликаем бутон. Варианти: `primary`, `secondary`, `ghost`, `destructive`, `link`.

Example.

```
button(етикет, onclick, вариант?)
```

`canvas`

Signature. `canvas(име, ш, в)`

Description. Чертожна повърхност вътре в прозорец.

Example.

```
canvas(име, ш, в)
```

`checkbox`

Signature. `checkbox(име, етикет)`

Description. Превключвател истина/неистина.

Example.

```
checkbox(име, етикет)
```

`column`

Signature. `column() ... endcolumn`

Description. Подрежда децата вертикално.

Example.

```
column() ... endcolumn
```

`confirm`

Signature. `confirm(съобщение, fn)`

Description. Диалог за потвърждение; извиква `fn` при OK.

Example.

```
confirm(съобщение, fn)
```

`divider`

Signature. `divider()`

Description. Хоризонтална разделителна линия.

Example.

```
divider()
```

`endwindow`

Signature. `endwindow`

Description. Затваря текущия блок прозорец.

Example.

```
endwindow
```

`group`

Signature. `group(заглавие) ... endgroup`

Description. Група с рамка и опционално заглавие.

Example.

```
group(заглавие) ... endgroup
```

`heading`

Signature. `heading(текст, ниво=1..6)`

Description. Заглавие в прозореца.

Example.

```
heading(текст, ниво=1..6)
```

`image`

Signature. `image(src, ш?, в?, alt?)`

Description. Изображение от локален път или URL.

Example.

```
image(src, ш?, в?, alt?)
```

`input`

Signature. `input(име, плейсхолдър?, стойност?)`

Description. Едноредово текстово поле, чиято стойност се чете с `uiget(име)`.

Example.

```
input(име, плейсхолдър?, стойност?)
```

`label`

Signature. `label(текст)`

Description. Кратък текстов етикет.

Example.

```
label(текст)
```

`map`

Signature. `map(шир, дълж, увеличение)`

Description. Вградена Leaflet карта.

Example.

```
map(шир, дълж, увеличение)
```

menu

Signature. `menu(t) ... endmenu / menuitem(t,fn)`

Description. Меню в горната част на прозореца.

Example.

```
menu(t) ... endmenu / menuitem(t,fn)
```

paragraph

Signature. `paragraph(текст)`

Description. Параграф основен текст.

Example.

```
paragraph(текст)
```

progress

Signature. `progress(стойност, макс?)`

Description. Лента за прогрес.

Example.

```
progress(стойност, макс?)
```

`prompt`

Signature. `prompt(съобщение, fn)`

Description. Диалог за въвеждане на текст.

Example.

```
prompt(съобщение, fn)
```

`radio`

Signature. `radio(име, опции)`

Description. Списък със взаимно изключващи се опции.

Example.

```
radio(име, опции)
```

`row`

Signature. `row() ... endrow`

Description. Подрежда децата хоризонтално.

Example.

```
row() ... endrow
```

`select`

Signature. `select(име, опции)`

Description. Падащ селектор.

Example.

```
select(име, опции)
```

`slider`

Signature. `slider(име, мин, макс, стъпка?)`

Description. Числов плъзгач.

Example.

```
slider(име, мин, макс, стъпка?)
```

`spacer`

Signature. `spacer(пиксели)`

Description. Празно вертикално разстояние.

Example.

```
spacer(пиксели)
```

`switch`

Signature. `switch(име, етикет)`

Description. Превключвател „включено/изключено“.

Example.

```
switch(име, етикет)
```

`table`

Signature. `table(заглавия, редове)`

Description. Таблица с данни.

Example.

```
table(заглавия, редове)
```

`tabs`

Signature. `tabs() ... endtabs / tab(т)..endtab`

Description. Контейнер с табове.

Example.

```
tabs() ... endtabs / tab(т)..endtab
```

`textarea`

Signature. `textarea(име, редове?, плейсхолдър?)`

Description. Многоредово текстово поле.

Example.

```
textarea(име, редове?, плейсхолдър?)
```

`toast`

Signature. `toast(съобщение, вид?)`

Description. Изскачащо известие, което не блокира.

Example.

```
toast(съобщение, вид?)
```

uiget

Signature. `uiget(име)`

Description. Чете стойността на компонент.

Example.

```
uiget(име)
```

uiset

Signature. `uiset(име, стойност)`

Description. Задава стойност на компонент програмно.

Example.

```
uiset(име, стойност)
```

video

Signature. `video(src, ш?, в?)`

Description. Вграден видео плейър.

Example.

```
video(src, ш?, в?)
```

`window`

Signature. `window(заглавие, ширина, височина)`

Description. Отваря прозорец от най-високо ниво. Затваря се с `endwindow`.

Example.

```
window(заглавие, ширина, височина)
```

ГЛАВА 09

Част III — Готварската книга

Част III — Готварската книга

Петнадесет урока стъпка по стъпка. Всеки е достатъчно кратък да се напише на ръка за 15–30 минути и достатъчно голям, за да научи реален урок.

Урок 1 — Здравей, прозорец

Цел. Отворете прозорец, покажете заглавие и параграф, реагирайте на бутон.

Стъпка 1 — Празният прозорец

```
window("Здравей", 360, 240)
endwindow
```

Стартирайте с **Ctrl + Enter**. Появява се прозорец 360×240. Празен е, защото нищо не сме сложили в него.

Стъпка 2 — Малко съдържание

```
window("Здравей", 360, 240)
  heading("Добре дошли в sdev", 1)
```

```
  paragraph("Това е възможно най-малката оконна програма.")
endwindow
```

Стъпка 3 — Бутон, който прави нещо

```
forge clicks be 0

window("Здравей", 360, 280)
  heading("Добре дошли в sdev", 1)
  paragraph("Натиснете бутона по-долу.")
  button("Натисни ме", () -&gt; {
    clicks = clicks + 1
    speak("кликнато " + clicks + " пъти")
  }, "primary")
endwindow
```

Стъпка 4 — Покажете брояча вътре в прозореца

```
window("Здравей", 360, 280)
  heading("Добре дошли в sdev", 1)
  paragraph("Натиснахте бутона " + uiget("count") + " пъти.")
  button("Натисни ме", () -&gt; uiset("count", (uiget("count") or (
endwindow
```

Какво научихте

- `window(...) ... endwindow` отваря прозорец.
- `heading`, `paragraph`, `button` са компоненти.
- `uiget` и `uiset` дават реактивно състояние.

Урок 2 — Калкулатор за бакшиш

Цел. Използвайте плъзгач, числово поле и изчислим параграф.

```
window("Бакшиш", 380, 320)
  heading("Калкулатор за бакшиш", 2)

  group("Сметка")
    input("amount", "0.00", "20.00", {type: "number", width: 160})
  endgroup

  group("Бакшиш")
    slider("tip", 0, 30, 1)
    paragraph("Бакшиш: " + uiget("tip") + "%")
  endgroup

  group("Резултат")
    forge total be elevate(uiget("amount") * (1 + uiget("tip")/100
    heading("Общо: " + total + " лв.", 3)
  endgroup
endwindow
```

`elevate(x, n)` закръгля до n знака след десетичната запетая.

Урок 3 — Списък със задачи

Цел. Рендериране на списък, добавяне, изтриване, запис локално.

```
forge todos be []
```

```
conjure addTodo():
  forge text be uiget("new")
  if text != ""
    todos = todos + [text]
    uiset("new", "")
    writeFile("todos.json", json.stringify(todos))
  end
end

conjure removeTodo(idx):
  todos = todos.removeAt(idx)
  writeFile("todos.json", json.stringify(todos))
end

attempt
  todos = json.parse(readFile("todos.json"))
rescue _
  todos = []
end

window("Задачи", 420, 540)
  heading("Моите задачи", 2)
  row({gap: 8})
    input("new", "Какво трябва да се свърши?", "", {width: 280})
    button("Добави", addTodo, "primary")
  endrow
  divider()
  for i in range(todos.length)
    row({gap: 8, align: "center"})
      paragraph((i+1) + ". " + todos[i])
      button("", () -&gt; removeTodo(i), "ghost", {icon: "trash"})
    endrow
  end
endwindow
```

Урок 4 — Приложение за бележки

Цел. Няколко бележки, превключване, автоматичен запис.

```
forge notes be tome()
forge current be "Без име"

attempt
  notes = json.parse(readFile("notes.json"))
rescue _
  notes["Без име"] = ""
end

conjure save():
  notes[current] = uiget("body")
  writeFile("notes.json", json.stringify(notes))
end

conjure switchTo(name):
  current = name
  uiset("body", notes[name])
end

window("Бележки", 720, 540)
  row({gap: 0})
    column({width: 200, background: "#0F172A", padding: 12})
      heading("Бележки", 4)
      for name in keys(notes)
        button(name, () -&gt; switchTo(name), "ghost", {width: 176
      end
      button("+ Нова", () -&gt; {
        notes["Бележка " + (keys(notes).length + 1)] = ""
        save()
      }, "primary", {width: 176})
    endcolumn
```

```
    column({padding: 12})
      heading(current, 3)
      textarea("body", 18, "Пишете тук...", {onChange: save})
    endcolumn
  endrow
endwindow
```

Урок 5 — Скицник

Цел. Компонентът `canvas`, събития от мишка, експорт като PNG.

```
window("Рисуване", 600, 480)
  heading("Скицник", 3)
  row({gap: 8})
    select("color", ["черно","червено","зелено","синьо","оранжево"
    slider("size", 1, 20, 1)
    button("Изчисти", () -&gt; canvasClear("pad"), "ghost")
    button("Запис PNG", () -&gt; canvasExport("pad", "drawing.png"
  endrow
  canvas("pad", 580, 380, {
    background: "#FFFFFF",
    border: "#CBD5E1",
    onMouseDrag: (x,y) -&gt; canvasDot("pad", x, y, uiget("size"),
  })
endwindow
```

Урок 6 — Помодоро таймер

Цел. Табове, лента за прогрес, аудио, настройки.

```
forge running be false
```

```
forge remaining be 25 * 60
forge totalLen be 25 * 60

conjure tick():
  if running and remaining &gt; 0
    remaining = remaining - 1
    uiset("progress", (totalLen - remaining) / totalLen * 100)
    after(1000, tick)
  elif remaining &lt;= 0
    audio("done.mp3")
    toast("Времето свърши!", "success")
    running = false
  end
end

window("Помодоро", 360, 360)
  tabs()
    tab("Таймер")
      heading(formatTime(remaining), 1)
      progress(uiget("progress") or 0, 100)
      row({gap: 8})
        button("Старт", () -&gt; { running = true ; tick() }, "pri
        button("Пауза", () -&gt; { running = false }, "secondary")
        button("Нулирай", () -&gt; { running = false ; remaining =
      endrow
    endtab
    tab("Настройки")
      slider("len", 5, 60, 5)
      button("Приложи", () -&gt; {
        totalLen = uiget("len") * 60
        remaining = totalLen
      }, "primary")
    endtab
  endtabs
endwindow
```

Урок 7 — Преглед на Markdown

Цел. Textarea + рендериран преглед един до друг.

```
window("Markdown", 900, 600)
  row({gap: 0})
    column({width: 440, padding: 8})
      heading("Източник", 4)
      textarea("md", 28, "# Здравей\n\nПишете **markdown** тук.",
    endcolumn
    column({padding: 16, background: "#FFFFFF"})
      heading("Преглед", 4)
      html(renderMarkdown(uiget("md") or ""))
    endcolumn
  endrow
endwindow
```

Урок 8 — Табло за времето

Цел. HTTP заявка, JSON, показване на отдалечени данни.

```
conjure load():
  forge city be uiget("city")
  forge resp be await http.get("https://api.example.com/weather?q=
  forge data be json.parse(resp.body)
  uiset("temp",     data.temp)
  uiset("humidity", data.humidity)
  uiset("desc",     data.description)
end

window("Времето", 380, 360)
```

```
  row({gap: 8})
    input("city", "Град", "София")
    button("Зареди", load, "primary")
  endrow
  group("Сега")
    heading(uiget("temp") + "°C", 1)
    paragraph(uiget("desc") or "—")
    paragraph("Влажност: " + (uiget("humidity") or "—") + "%")
  endgroup
endwindow
```

Урок 9 — Карта-изследовател

Цел. Leaflet карта, маркери, търсене.

```
forge places be [
  ["София", 42.6977, 23.3219],
  ["Пловдив", 42.1354, 24.7453],
  ["Варна", 43.2141, 27.9147]
]

window("Карта", 720, 520)
  input("q", "Търсене на място")
  map("m", 42.7, 25.0, 7, {width: 700, height: 420})
  for p in places
    marker("m", p[1], p[2], p[0])
  end
  button("Намери", () -&gt; {
    forge q be lower(uiget("q"))
    for p in places
      if contains(lower(p[0]), q)
        flyTo("m", p[1], p[2], 11)
        escape
      end
```

```
    end
  }, "primary")
endwindow
```

Урок 10 — Мини електронна таблица

Цел. Редактируема таблица с малък evaluator на формули.

```
forge cells be matrix(8, 5, () -&gt; "")

conjure setCell(r, c, v):
  cells[r][c] = v
end

conjure render(v):
  if startsWith(v, "=")
    return eval(substring(v, 1))
  end
  return v
end

window("Таблица", 640, 420)
  heading("Мини таблица", 3)
  for r in range(8)
    row({gap: 4})
      for c in range(5)
        input("c" + r + "_" + c, "", cells[r][c], {
          width: 110,
          onChange: () -&gt; setCell(r, c, uiget("c" + r + "_" + c
        })
      end
    endrow
  end
```

```
  divider()
  paragraph("Сума на A1: " + render(cells[0][0]))
endwindow
```

Урок 11 — Прозорец за чат

Цел. Скрол, async поток, балончета със съобщения.

```
forge messages be []

conjure send():
  forge msg be uiget("input")
  if msg == "" then return end
  messages = messages + [{from: "me", text: msg}]
  uiset("input", "")
  await stream("https://api.example.com/chat", msg, (chunk) -&gt; {
    if last(messages).from != "bot"
      messages = messages + [{from: "bot", text: ""}]
    end
    last(messages).text = last(messages).text + chunk
  })
end

window("Чат", 480, 600)
  scrollarea({stickToBottom: true, height: 480})
    for m in messages
      paragraph(m.text, {
        background: m.from == "me" ? "#0891B2" : "#1E293B",
        foreground: "#FFFFFF",
        padding: 10,
        radius: 12,
        align: m.from == "me" ? "end" : "start"
      })
    end
  endscrollarea
  row({gap: 8})
```

```
    input("input", "Кажете нещо…")
    button("Изпрати", send, "primary")
  endrow
endwindow
```

Урок 12 — Файлов мениджър

Цел. Дърво на работното пространство, разгръщане при клик.

```
conjure renderNode(node, depth):
  for child in node.children
    row({gap: 4, padding: depth * 12})
      if child.type == "folder"
        button("▸ " + child.name, () -&gt; child.expanded = not chi
      else
        button("▫ " + child.name, () -&gt; openFile(child.path), "
      end
    endrow
    if child.type == "folder" and child.expanded
      renderNode(child, depth + 1)
    end
  end
end

window("Файлове", 360, 600)
  heading("Работно пространство", 3)
  forge tree be readWorkspaceTree()
  renderNode(tree, 0)
endwindow
```

Урок 13 — Калкулатор

Цел. Grid оформление, варианти бутони, бързи клавиши.

```
forge expr be ""

conjure press(key):
  if key == "C" then expr = ""
  elif key == "=" then expr = "" + eval(expr)
  else expr = expr + key end
  uiset("display", expr)
end

window("Калкулатор", 280, 360)
  paragraph(uiget("display") or "0", {
    background: "#0F172A", foreground: "#F8FAFC",
    padding: 14, radius: 8, align: "end", monospace: true
  })
  for row in [["7","8","9","/"],["4","5","6","*"],["1","2","3","-"
    row({gap: 6})
      for k in row
        button(k, () -&gt; press(k), k == "=" ? "primary" : "second
               {width: 56, height: 48})
      end
    endrow
  end
  button("Изчисти", () -&gt; press("C"), "destructive", {width: 240
endwindow
```

Урок 14 — Подскачаща топка

Цел. Анимационен цикъл върху canvas със сблъсъци.

```
forge x be 100; forge y be 100
forge vx be 3;  forge vy be 2

conjure frame():
  canvasClear("g", "#0F172A")
  canvasCircle("g", x, y, 16, "#0891B2")
  x = x + vx ; y = y + vy
  if x &lt; 16 or x &gt; 484 then vx = -vx end
  if y &lt; 16 or y &gt; 284 then vy = -vy end
  after(16, frame)
end

window("Топка", 520, 340)
  canvas("g", 500, 300)
  frame()
endwindow
```

Урок 15 — Вашата първа библиотека

Цел. Пакетирайте полезен код в Gist и го извикайте с `summon`.

Стъпка 1 — Напишете библиотеката

Файл `mylib.sdev`:

```
conjure greet(who):
  return "Здравей, " + who + "!"
end
```

Стъпка 2 — Публикувайте като публичен Gist

В IDE: **File → Publish as Gist**. Копирайте URL.

Стъпка 3 — Използвайте я

```
summon "https://gist.github.com/&lt;user&gt;/&lt;hash&gt;" as mylib

speak(mylib.greet("свят"))
```

Накъде нататък

Видяхте всяка част от езика и инструментариума. Изберете проект, който ви вълнува, отворете IDE и започнете да пишете.

ГЛАВА 10

Приложение A — Карти и GIS (Leaflet)

Приложение A — Карти и GIS (Leaflet)

Пълен справочник за картографската подсистема (на английски — имената на функциите са еднакви и в двата езика).

ГЛАВА 11

sdev Leaflet Documentation

sdev Leaflet Documentation

Interactive Maps for sdev

The sdev Leaflet module provides powerful geographic mapping capabilities, allowing you to create interactive maps, markers, shapes, and more using the familiar sdev syntax.

Table of Contents

1. Getting Started
2. Map Creation
3. Markers & Popups
4. Shapes
5. Polylines & Polygons
6. Layers
7. Events
8. Controls
9. GeoJSON
10. Utilities
11. Complete Examples

Getting Started

Basic Setup

To use Leaflet features in sdev, first create a map container:

```
// Create a map centered on coordinates with zoom level
forge myMap be createMap("map-container", 51.505, -0.09, 13)
```

HTML Setup

Your HTML page needs a container div and Leaflet CSS/JS:

```
&lt;!DOCTYPE html&gt;
&lt;html&gt;
&lt;head&gt;
    &lt;link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4
    &lt;script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js
    &lt;script src="sdev-interpreter.js"&gt;&lt;/script&gt;
    &lt;style&gt;
        #map-container { height: 500px; width: 100%; }
    &lt;/style&gt;
&lt;/head&gt;
&lt;body&gt;
    &lt;div id="map-container"&gt;&lt;/div&gt;
    &lt;script&gt;
        const interpreter = new SdevInterpreter();
        interpreter.run(`
            forge map be createMap("map-container", 51.505, -0.09,
            addMarker(map, 51.505, -0.09, "Hello from sdev!")
        `);
    &lt;/script&gt;
&lt;/body&gt;
```

```
&lt;/html&gt;
```

Map Creation

createMap(containerId, lat, lng, zoom)

Creates a new Leaflet map instance.

| Parameter | Type | Description | |----------|------|----------| | containerId | text | ID of the HTML container element | | lat | number | Initial latitude center | | lng | number | Initial longitude center | | zoom | number | Initial zoom level (1-18) |

```
// Create a map of London
forge londonMap be createMap("map", 51.505, -0.09, 13)

// Create a map of New York
forge nyMap be createMap("nyc-map", 40.7128, -74.0060, 12)
```

setMapView(map, lat, lng, zoom)

Changes the map's center and zoom level.

```
forge map be createMap("map", 0, 0, 2)

// Pan to Paris
setMapView(map, 48.8566, 2.3522, 14)
```

getMapCenter(map)

Returns the current center coordinates as a tome (dictionary).

```
forge center be getMapCenter(map)
speak("Lat: " + morph(center["lat"], "text"))
speak("Lng: " + morph(center["lng"], "text"))
```

getMapZoom(map)

Returns the current zoom level.

```
forge zoom be getMapZoom(map)
speak("Current zoom: " + morph(zoom, "text"))
```

getMapBounds(map)

Returns the visible map bounds.

```
forge bounds be getMapBounds(map)
// bounds contains: north, south, east, west
```

Markers & Popups

addMarker(map, lat, lng, popupText?)

Adds a marker to the map with an optional popup.

```
forge map be createMap("map", 51.505, -0.09, 13)

// Simple marker
forge marker1 be addMarker(map, 51.505, -0.09)
```

```
// Marker with popup
forge marker2 be addMarker(map, 51.51, -0.08, "Click me!")
```

addMarkerIcon(map, lat, lng, iconUrl, iconSize, popupText?)

Adds a marker with a custom icon.

| Parameter | Type | Description |
|----------|------|----------| | iconUrl | text | URL to the icon image | | iconSize | list | [width, height] in pixels |

```
forge map be createMap("map", 51.505, -0.09, 13)

forge customMarker be addMarkerIcon(
    map,
    51.505,
    -0.09,
    "https://example.com/pin.png",
    [32, 32],
    "Custom icon marker!"
)
```

removeMarker(map, marker)

Removes a marker from the map.

```
forge marker be addMarker(map, 51.505, -0.09, "Temporary")
// Later...
removeMarker(map, marker)
```

setMarkerPosition(marker, lat, lng)

Moves an existing marker to new coordinates.

```
forge marker be addMarker(map, 51.505, -0.09)

// Animate marker movement
forge i be 0
cycle i &lt; 100 ::
    setMarkerPosition(marker, 51.505 + i * 0.001, -0.09 + i * 0.00
    delay(50)
    i be i + 1
;;
```

bindPopup(marker, content)

Attaches a popup to an existing marker.

```
forge marker be addMarker(map, 51.505, -0.09)
bindPopup(marker, "&lt;b&gt;Bold text!&lt;/b&gt;&lt;br&gt;HTML wor
```

bindTooltip(marker, content)

Attaches a tooltip (shows on hover) to a marker.

```
forge marker be addMarker(map, 51.505, -0.09)
bindTooltip(marker, "Hover tooltip")
```

openPopup(marker)

Programmatically opens the marker's popup.

```
forge marker be addMarker(map, 51.505, -0.09, "Hello!")
openPopup(marker)
```

Shapes

addCircle(map, lat, lng, radius, options?)

Adds a circle to the map.

Parameter	Type	Description
radius	number	Radius in meters
options	tome	Style options (optional)

```
forge map be createMap("map", 51.505, -0.09, 13)

// Simple circle
forge circle1 be addCircle(map, 51.505, -0.09, 500)

// Styled circle
forge options be ::
    "color": "#ff0000",
    "fillColor": "#ff6666",
    "fillOpacity": 0.5,
    "weight": 2
;;
forge circle2 be addCircle(map, 51.51, -0.08, 300, options)
```

addCircleMarker(map, lat, lng, radius, options?)

Adds a circle marker (radius in pixels, not meters).

```
forge dot be addCircleMarker(map, 51.505, -0.09, 10, ::
    "color": "#3388ff",
    "fillColor": "#3388ff",
    "fillOpacity": 0.8
```

```
;;)
```

addRectangle(map, lat1, lng1, lat2, lng2, options?)

Adds a rectangle defined by opposite corners.

```
forge rect be addRectangle(
    map,
    51.49, -0.10,  // Southwest corner
    51.52, -0.06,  // Northeast corner
    :: "color": "#ff7800", "weight": 1 ;;
)
```

Polylines & Polygons

addPolyline(map, points, options?)

Draws a line through multiple points.

| Parameter | Type | Description | |----------|------|----------| | points | list | List of [lat, lng] coordinate pairs | | options | tome | Style options |

```
forge map be createMap("map", 51.505, -0.09, 13)

forge route be [
    [51.505, -0.09],
    [51.51, -0.08],
    [51.52, -0.06],
    [51.515, -0.05]
]
```

```
forge line be addPolyline(map, route, ::
    "color": "#ff0000",
    "weight": 4,
    "opacity": 0.8,
    "dashArray": "10, 10"
;;)
```

addPolygon(map, points, options?)

Creates a closed polygon shape.

```
forge triangle be [
    [51.509, -0.08],
    [51.503, -0.06],
    [51.51, -0.047]
]

forge poly be addPolygon(map, triangle, ::
    "color": "#00ff00",
    "fillColor": "#00ff88",
    "fillOpacity": 0.4
;;)
```

addMultiPolygon(map, polygons, options?)

Creates multiple polygons as a single layer.

```
forge shapes be [
    [[51.51, -0.12], [51.51, -0.10], [51.52, -0.10], [51.52, -0.12
    [[51.51, -0.08], [51.51, -0.06], [51.52, -0.06], [51.52, -0.08
]

forge multiPoly be addMultiPolygon(map, shapes, ::
    "color": "#9900ff"
;;)
```

Layers

addTileLayer(map, urlTemplate, options?)

Adds a custom tile layer (base map).

```
forge map be createMap("map", 51.505, -0.09, 13)

// OpenStreetMap (default)
addTileLayer(map, "https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.
    "attribution": "© OpenStreetMap contributors"
;;)

// Satellite imagery
addTileLayer(map, "https://server.arcgisonline.com/ArcGIS/rest/serv
    "attribution": "© Esri"
;;)
```

createLayerGroup()

Creates an empty layer group for organizing layers.

```
forge markers be createLayerGroup()
addMarker(markers, 51.505, -0.09, "Marker 1")
addMarker(markers, 51.51, -0.08, "Marker 2")
addLayerToMap(map, markers)
```

addLayerToMap(map, layer)

Adds a layer or layer group to the map.

```
forge group be createLayerGroup()
// Add items to group...
addLayerToMap(map, group)
```

removeLayerFromMap(map, layer)

Removes a layer from the map.

```
removeLayerFromMap(map, markers)
```

clearLayer(layer)

Removes all items from a layer group.

```
clearLayer(markers)
```

Events

onMapClick(map, callback)

Handles map click events.

```
forge map be createMap("map", 51.505, -0.09, 13)

onMapClick(map, (event) -&gt; ::
    forge lat be event["lat"]
    forge lng be event["lng"]
    addMarker(map, lat, lng, "Clicked at " + morph(lat, "text") +
;;)
```

onMapZoom(map, callback)

Handles zoom changes.

```
onMapZoom(map, (event) -&gt; ::
    forge zoom be getMapZoom(map)
    speak("Zoom changed to: " + morph(zoom, "text"))
;;)
```

onMapMove(map, callback)

Handles map movement (pan).

```
onMapMove(map, (event) -&gt; ::
    forge center be getMapCenter(map)
    speak("Map moved to: " + morph(center["lat"], "text") + ", " +
;;)
```

onMarkerClick(marker, callback)

Handles marker click events.

```
forge marker be addMarker(map, 51.505, -0.09)

onMarkerClick(marker, (event) -&gt; ::
    speak("Marker was clicked!")
;;)
```

onMarkerDrag(marker, callback)

Handles marker drag events (marker must be draggable).

```
forge marker be addMarker(map, 51.505, -0.09)
```

```
setMarkerDraggable(marker, yep)

onMarkerDrag(marker, (event) -&gt; ::
    forge pos be getMarkerPosition(marker)
    speak("Dragged to: " + morph(pos["lat"], "text"))
;;)
```

Controls

addZoomControl(map, position?)

Adds zoom controls to the map.

| Position | Description | |---------|----------| | "topleft" | Top left corner | | "topright" | Top right corner | | "bottomleft" | Bottom left corner | | "bottomright" | Bottom right corner |

```
addZoomControl(map, "bottomright")
```

addScaleControl(map, options?)

Adds a scale indicator.

```
addScaleControl(map, ::
    "position": "bottomleft",
    "metric": yep,
    "imperial": nope
;;)
```

addAttributionControl(map, prefix?)

Adds attribution text.

```
addAttributionControl(map, "Powered by sdev")
```

addLayerControl(map, baseLayers, overlays)

Adds a layer switcher control.

```
forge osm be addTileLayer(map, "https://{s}.tile.openstreetmap.org/
forge satellite be addTileLayer(map, "https://server.arcgisonline.c

forge bases be ::
    "Streets": osm,
    "Satellite": satellite
;;

forge markers be createLayerGroup()
forge overlays be ::
    "Markers": markers
;;

addLayerControl(map, bases, overlays)
```

GeoJSON

addGeoJSON(map, geoJsonData, options?)

Adds GeoJSON data to the map.

```
forge geojson be ::
```

```
    "type": "FeatureCollection",
    "features": [
        ::
            "type": "Feature",
            "geometry": ::
                "type": "Point",
                "coordinates": [-0.09, 51.505]
            ;;,
            "properties": ::
                "name": "London"
            ;;
        ;;
    ]
;;

forge layer be addGeoJSON(map, geojson, ::
    "style": ::
        "color": "#ff0000",
        "weight": 2
    ;;
;;)
```

geoJSONStyle(feature)

Custom style function for GeoJSON features.

```
forge layer be addGeoJSON(map, geojson, ::
    "style": (feature) -&gt; ::
        ponder feature["properties"]["type"] equals "road" ::
            yield :: "color": "#888888", "weight": 3 ;;
        ;;
        otherwise ::
            yield :: "color": "#00ff00", "weight": 1 ;;
        ;;
    ;;
;;)
```

Utilities

latLng(lat, lng)

Creates a coordinate object.

```
forge coord be latLng(51.505, -0.09)
speak("Latitude: " + morph(coord["lat"], "text"))
```

distance(lat1, lng1, lat2, lng2)

Calculates distance between two points in meters.

```
forge dist be distance(51.505, -0.09, 51.51, -0.08)
speak("Distance: " + morph(dist, "text") + " meters")
```

boundsContains(bounds, lat, lng)

Checks if a point is within bounds.

```
forge bounds be getMapBounds(map)
forge inside be boundsContains(bounds, 51.505, -0.09)
ponder inside ::
    speak("Point is visible on map")
;;
```

fitBounds(map, lat1, lng1, lat2, lng2)

Adjusts the map view to fit the given bounds.

```
fitBounds(map, 51.49, -0.12, 51.52, -0.05)
```

invalidateSize(map)

Recalculates map size (use after container resize).

```
invalidateSize(map)
```

Navigation & Animation

panTo(map, lat, lng, options?)

Smoothly pans the map to a new center.

```
panTo(map, 48.8566, 2.3522)
```

panBy(map, x, y)

Pans the map by a given number of pixels.

```
panBy(map, 100, 50)  // Pan right 100px, down 50px
```

flyTo(map, lat, lng, zoom, options?)

Animates the map to a new position with a smooth flying effect.

```
flyTo(map, 40.7128, -74.0060, 14)  // Fly to New York
```

flyToBounds(map, lat1, lng1, lat2, lng2, options?)

Animates to fit the given bounds.

```
flyToBounds(map, 51.49, -0.12, 51.52, -0.05)
```

zoomIn(map, delta?)

Increases the zoom level by delta (default 1).

```
zoomIn(map)        // Zoom in by 1
zoomIn(map, 2)     // Zoom in by 2
```

zoomOut(map, delta?)

Decreases the zoom level by delta (default 1).

```
zoomOut(map)
```

setZoom(map, zoom)

Sets the zoom level directly.

```
setZoom(map, 15)
```

setMinZoom(map, zoom) / setMaxZoom(map, zoom)

Sets zoom constraints.

```
setMinZoom(map, 5)
setMaxZoom(map, 18)
```

getMinZoom(map) / getMaxZoom(map)

Gets the current zoom constraints.

```
forge minZ be getMinZoom(map)
forge maxZ be getMaxZoom(map)
```

setMaxBounds(map, lat1, lng1, lat2, lng2)

Restricts the map to a given geographical area.

```
setMaxBounds(map, 51.0, -0.5, 52.0, 0.5)  // Lock to London area
```

Popup & Tooltip Control

closePopup(marker)

Closes an open popup.

```
closePopup(marker)
```

openTooltip(marker) / closeTooltip(marker)

Opens or closes a marker's tooltip.

```
openTooltip(marker)
closeTooltip(marker)
```

setPopupContent(marker, content)

Updates popup content dynamically.

```
setPopupContent(marker, "Updated content!")
```

setTooltipContent(marker, content)

Updates tooltip content dynamically.

```
setTooltipContent(marker, "New tooltip text")
```

Layer Styling

setMarkerIcon(marker, iconUrl, iconSize)

Changes a marker's icon.

```
setMarkerIcon(marker, "https://example.com/new-icon.png", [32, 32]
```

setMarkerOpacity(marker, opacity)

Sets marker transparency (0-1).

```
setMarkerOpacity(marker, 0.5)
```

setMarkerZIndex(marker, zIndex)

Controls marker stacking order.

```
setMarkerZIndex(marker, 1000)
```

setCircleRadius(circle, radius)

Updates a circle's radius in meters.

```
setCircleRadius(myCircle, 750)
```

setCircleStyle(circle, options) / setPolylineStyle / setPolygonStyle

Updates styling of shapes.

```
setCircleStyle(myCircle, :: "color": "#ff0000", "fillOpacity": 0.8
setPolylineStyle(myLine, :: "color": "#00ff00", "weight": 5 ;;)
setPolygonStyle(myPoly, :: "fillColor": "#0000ff" ;;)
```

getPolylineLatLngs(polyline)

Gets all points of a polyline.

```
forge points be getPolylineLatLngs(myRoute)
```

setPolylineLatLngs(polyline, points)

Replaces all points of a polyline.

```
setPolylineLatLngs(myRoute, [[51.5, -0.1], [51.6, -0.05]])
```

addLatLng(polyline, lat, lng)

Adds a point to a polyline.

```
addLatLng(myRoute, 51.52, -0.08)
```

bringToFront(layer) / bringToBack(layer)

Controls layer z-ordering.

```
bringToFront(importantLayer)
bringToBack(backgroundLayer)
```

Additional Events

onLayerClick(layer, callback)

Handles click events on any layer.

```
onLayerClick(myCircle, (e) -&gt; ::
    speak("Circle clicked at " + morph(e["lat"], "text"))
;;)
```

onLayerMouseover(layer, callback) / onLayerMouseout(layer, callback)

Handles hover events.

```
onLayerMouseover(myPolygon, (e) -&gt; ::
```

```
    setPolygonStyle(myPolygon, :: "fillOpacity": 0.8 ;;)
;;)

onLayerMouseout(myPolygon, (e) -&gt; ::
    setPolygonStyle(myPolygon, :: "fillOpacity": 0.4 ;;)
;;)
```

onMapDoubleClick(map, callback)

Handles double-click events.

```
onMapDoubleClick(map, (e) -&gt; ::
    setMapView(map, e["lat"], e["lng"], getMapZoom(map) + 1)
;;)
```

onMapRightClick(map, callback)

Handles right-click (context menu) events.

```
onMapRightClick(map, (e) -&gt; ::
    speak("Right click at " + morph(e["lat"], "text"))
;;)
```

onMapMousemove(map, callback)

Tracks mouse movement over the map.

```
onMapMousemove(map, (e) -&gt; ::
    updateCoordinateDisplay(e["lat"], e["lng"])
;;)
```

Geolocation

locate(map, options?)

Starts browser geolocation.

```
locate(map, :: "setView": yep, "maxZoom": 16 ;;)
```

onLocationFound(map, callback)

Handles successful location.

```
onLocationFound(map, (e) -&gt; ::
    speak("Found you at " + morph(e["lat"], "text") + ", " + morph
    addMarker(map, e["lat"], e["lng"], "You are here!")
;;)
```

onLocationError(map, callback)

Handles geolocation errors.

```
onLocationError(map, (e) -&gt; ::
    speak("Location error: " + e["message"])
;;)
```

stopLocate(map)

Stops continuous location updates.

```
stopLocate(map)
```

Overlays

addImageOverlay(map, imageUrl, lat1, lng1, lat2, lng2, options?)

Adds an image overlay to the map.

```
forge overlay be addImageOverlay(map, "historical-map.jpg", 51.4,
```

addVideoOverlay(map, videoUrl, lat1, lng1, lat2, lng2, options?)

Adds a video overlay.

```
forge video be addVideoOverlay(map, "timelapse.mp4", 51.4, -0.2, 5
```

setImageOpacity(overlay, opacity)

Sets overlay transparency.

```
setImageOpacity(overlay, 0.7)
```

setImageUrl(overlay, url)

Changes the overlay image.

```
setImageUrl(overlay, "new-image.jpg")
```

setBounds(overlay, lat1, lng1, lat2, lng2)

Repositions an overlay.

```
setBounds(overlay, 51.3, -0.3, 51.7, 0.2)
```

Feature Groups

createFeatureGroup()

Creates a feature group (like layer group but with bounds).

```
forge group be createFeatureGroup()
```

addToFeatureGroup(featureGroup, layer)

Adds a layer to the feature group.

```
addToFeatureGroup(group, myMarker)
addToFeatureGroup(group, myCircle)
```

removeFromFeatureGroup(featureGroup, layer)

Removes a layer from the feature group.

```
removeFromFeatureGroup(group, myMarker)
```

getFeatureGroupBounds(featureGroup)

Gets the combined bounds of all layers.

```
forge bounds be getFeatureGroupBounds(group)
```

fitFeatureGroup(map, featureGroup, options?)

Zooms the map to fit all layers in the group.

```
fitFeatureGroup(map, group)
```

eachLayer(layerGroup, callback)

Iterates over all layers in a group.

```
eachLayer(group, (layer) -&gt; ::
    speak("Processing layer")
;;)
```

getLayers(layerGroup)

Returns all layers as a list.

```
forge layers be getLayers(group)
```

hasLayer(layerGroup, layer)

Checks if a layer exists in the group.

```
ponder hasLayer(group, myMarker) ::
    speak("Marker is in group")
;;
```

Custom Markers

addDivIcon(map, lat, lng, html, className, size)

Creates a marker with custom HTML content.

```
forge customMarker be addDivIcon(map, 51.505, -0.09, "&lt;div class
```

Coordinate Utilities

getSize(map)

Gets map container size in pixels.

```
forge size be getSize(map)
speak("Width: " + morph(size["width"], "text") + ", Height: " + mo
```

latLngToContainerPoint(map, lat, lng)

Converts coordinates to pixel position.

```
forge pixel be latLngToContainerPoint(map, 51.505, -0.09)
speak("Pixel X: " + morph(pixel["x"], "text"))
```

containerPointToLatLng(map, x, y)

Converts pixel position to coordinates.

```
forge coord be containerPointToLatLng(map, 200, 150)
```

wrapLng(lng)

Wraps longitude to -180 to 180.

```
forge wrapped be wrapLng(370)  // Returns 10
```

wrapLat(lat)

Clamps latitude to -90 to 90.

```
forge clamped be wrapLat(95)  // Returns 90
```

degreesToDMS(degrees)

Converts decimal degrees to DMS string.

```
forge dms be degreesToDMS(51.5074)  // "51° 30' 26.64""
```

DMSToDegrees(d, m, s)

Converts DMS to decimal degrees.

```
forge deg be DMSToDegrees(51, 30, 26.64)  // 51.5074
```

metersToPixels(map, meters, lat)

Converts meters to pixels at the current zoom.

```
forge px be metersToPixels(map, 1000, 51.505)
```

pixelsToMeters(map, pixels, lat)

Converts pixels to meters at the current zoom.

```
forge m be pixelsToMeters(map, 100, 51.505)
```

GIS Analysis Functions

bearing(lat1, lng1, lat2, lng2)

Calculates bearing between two points (0-360 degrees).

```
forge b be bearing(51.5, -0.1, 48.8, 2.3)  // London to Paris
speak("Bearing: " + morph(b, "text") + "°")
```

midpoint(lat1, lng1, lat2, lng2)

Calculates the midpoint between two coordinates.

```
forge mid be midpoint(51.5, -0.1, 48.8, 2.3)
addMarker(map, mid["lat"], mid["lng"], "Midpoint")
```

destination(lat, lng, bearing, distance)

Calculates destination point given bearing and distance.

```
forge dest be destination(51.5, -0.1, 90, 50000)  // 50km east of l
addMarker(map, dest["lat"], dest["lng"], "Destination")
```

area(points)

Calculates polygon area in square meters.

```
forge polygon be [[51.5, -0.1], [51.5, 0.0], [51.4, 0.0], [51.4, -0
forge sqMeters be area(polygon)
speak("Area: " + morph(ground(sqMeters / 1000000), "text") + " km²
```

length(points)

Calculates polyline length in meters.

```
forge route be [[51.5, -0.1], [51.52, -0.08], [51.55, -0.05]]
forge len be length(route)
speak("Route length: " + morph(ground(len), "text") + " meters")
```

centroid(points)

Calculates the center point of a polygon.

```
forge center be centroid(polygon)
addMarker(map, center["lat"], center["lng"], "Center")
```

isPointInPolygon(lat, lng, points)

Checks if a point is inside a polygon.

```
ponder isPointInPolygon(51.45, -0.05, polygon) ::
    speak("Point is inside the polygon")
;; otherwise ::
    speak("Point is outside")
;;
```

simplify(points, tolerance)

Simplifies a polyline using Douglas-Peucker algorithm.

```
forge simplified be simplify(complexRoute, 0.0001)
```

interpolateAlong(points, fraction)

Gets a point at a fraction (0-1) along a polyline.

```
forge halfway be interpolateAlong(route, 0.5)
addMarker(map, halfway["lat"], halfway["lng"], "Halfway point")
```

Layer Visibility

getLayerType(layer)

Gets the type of a layer.

```
forge type be getLayerType(myLayer)  // "marker", "circle", etc.
```

isLayerVisible(layer)

Checks if a layer is currently on the map.

```
ponder isLayerVisible(myMarker) ::
    speak("Marker is visible")
;;
```

showLayer(map, layer) / hideLayer(map, layer)

Shows or hides a layer.

```
hideLayer(map, secretMarker)
// Later...
showLayer(map, secretMarker)
```

toggleLayer(map, layer)

Toggles layer visibility.

```
toggleLayer(map, myLayer)
```

Complete Examples

Interactive City Markers

```
// Create a world map with major cities
forge map be createMap("map", 20, 0, 2)

// City data
forge cities be [
    :: "name": "London", "lat": 51.505, "lng": -0.09 ;;,
    :: "name": "Paris", "lat": 48.8566, "lng": 2.3522 ;;,
    :: "name": "New York", "lat": 40.7128, "lng": -74.0060 ;;,
    :: "name": "Tokyo", "lat": 35.6762, "lng": 139.6503 ;;,
    :: "name": "Sydney", "lat": -33.8688, "lng": 151.2093 ;;
]

// Add markers for each city
```

```
each(cities, (city) -&gt; ::
    forge marker be addMarker(map, city["lat"], city["lng"], city[
    bindTooltip(marker, city["name"])
;;)

// Click to add new markers
onMapClick(map, (e) -&gt; ::
    addMarker(map, e["lat"], e["lng"], "New Location")
;;)
```

Route Visualization

```
// Visualize a hiking route
forge map be createMap("map", 51.505, -0.09, 14)

forge trailPoints be [
    [51.500, -0.10],
    [51.502, -0.095],
    [51.505, -0.09],
    [51.508, -0.085],
    [51.510, -0.08],
    [51.512, -0.075]
]

// Draw the route
forge trail be addPolyline(map, trailPoints, ::
    "color": "#e74c3c",
    "weight": 5,
    "opacity": 0.8
;;)

// Add start and end markers
addMarker(map, 51.500, -0.10, "□ Start")
addMarker(map, 51.512, -0.075, "□ Finish")

// Add distance markers along the way
forge totalDist be 0
forge i be 1
```

```
cycle i &lt; measure(trailPoints) ::
    forge prev be pluck(trailPoints, i - 1)
    forge curr be pluck(trailPoints, i)
    forge dist be distance(prev[0], prev[1], curr[0], curr[1])
    totalDist be totalDist + dist

    addCircleMarker(map, curr[0], curr[1], 5, ::
        "color": "#3498db",
        "fillColor": "#3498db",
        "fillOpacity": 1
    ;;)
    i be i + 1
;;

speak("Total distance: " + morph(ground(totalDist), "text") + " met
```

Heatmap Zones

```
// Create density visualization with circles
forge map be createMap("map", 51.505, -0.09, 13)

// Data points with intensity
forge hotspots be [
    :: "lat": 51.505, "lng": -0.09, "intensity": 100 ;;,
    :: "lat": 51.510, "lng": -0.08, "intensity": 75 ;;,
    :: "lat": 51.500, "lng": -0.10, "intensity": 50 ;;,
    :: "lat": 51.508, "lng": -0.095, "intensity": 90 ;;
]

// Create gradient circles for each hotspot
each(hotspots, (spot) -&gt; ::
    // Outer glow
    addCircle(map, spot["lat"], spot["lng"], spot["intensity"] * 5
        "color": "transparent",
        "fillColor": "#ff6600",
        "fillOpacity": 0.2
    ;;)
```

```
    // Inner core
    addCircle(map, spot["lat"], spot["lng"], spot["intensity"] * 2
        "color": "transparent",
        "fillColor": "#ff0000",
        "fillOpacity": 0.5
    ;;)
;;)
```

Layer Toggle System

```
// Multi-layer map with toggle controls
forge map be createMap("map", 51.505, -0.09, 13)

// Create layer groups
forge restaurants be createLayerGroup()
forge hotels be createLayerGroup()
forge attractions be createLayerGroup()

// Add restaurant markers
addMarker(restaurants, 51.505, -0.09, "Pizza Place")
addMarker(restaurants, 51.508, -0.085, "Sushi Bar")
addMarker(restaurants, 51.502, -0.095, "Burger Joint")

// Add hotel markers
addMarker(hotels, 51.510, -0.08, "Grand Hotel")
addMarker(hotels, 51.500, -0.10, "Budget Inn")

// Add attraction markers
addMarker(attractions, 51.507, -0.09, "Museum")
addMarker(attractions, 51.503, -0.07, "Park")

// Add all layers to map
addLayerToMap(map, restaurants)
addLayerToMap(map, hotels)
addLayerToMap(map, attractions)

// Create layer control
forge overlays be ::
```

```
    "□ Restaurants": restaurants,
    "□ Hotels": hotels,
    "□ Attractions": attractions
;;

addLayerControl(map, ::;;, overlays)
```

Animated Marker

```
// Animate a marker along a path
forge map be createMap("map", 51.505, -0.09, 14)

forge path be [
    [51.500, -0.10],
    [51.502, -0.095],
    [51.505, -0.09],
    [51.508, -0.085],
    [51.510, -0.08]
]

// Draw the path
addPolyline(map, path, :: "color": "#3498db", "weight": 3, "dashAr

// Create moving marker
forge mover be addMarker(map, path[0][0], path[0][1], "□")

// Animate along path
conjure animateMarker(marker, points, index) ::
    ponder index &gt;= measure(points) ::
        yield void
    ;;

    forge target be pluck(points, index)
    setMarkerPosition(marker, target[0], target[1])

    delay(500)
    animateMarker(marker, points, index + 1)
;;
```

```
animateMarker(mover, path, 0)
```

Style Reference

Common Style Options

| Property | Type | Description | |----------|------|----------| | color | text | Stroke color (hex or name) | | weight | number | Stroke width in pixels | | opacity | number | Stroke opacity (0-1) | | fillColor | text | Fill color | | fillOpacity | number | Fill opacity (0-1) | | dashArray | text | Stroke dash pattern | | lineCap | text | Line cap style | | lineJoin | text | Line join style |

Icon Options

| Property | Type | Description | |----------|------|----------| | iconUrl | text | URL to icon image | | iconSize | list | [width, height] | | iconAnchor | list | [x, y] anchor point | | popupAnchor | list | [x, y] popup offset | | shadowUrl | text | URL to shadow image |

Tips & Best Practices

1. **Performance**: Use layer groups for many markers
2. **Memory**: Remove unused layers with `removeLayerFromMap`

3. **Mobile**: Use `invalidateSize` after orientation changes
4. **Clustering**: For 100+ markers, consider marker clustering
5. **Tile Caching**: Custom tile layers can be cached for offline use

Error Handling

```
attempt ::
    forge map be createMap("nonexistent-id", 0, 0, 10)
;; rescue error ::
    speak("Failed to create map: " + error)
;;
```

sdev Leaflet Module — Mapping made magical □□

ГЛАВА 12

Приложение Б — Речник

Приложение Б — Речник

A quick lookup of every keyword and built-in name introduced in this book.

| Symbol | Meaning | |--------|---------| | `after(ms, fn)` | Изпълнява `fn` след `ms` милисекунди. | | `attempt … rescue` | Блок за обработка на грешки. | | `audio(src)` | Пуска аудио файл или URL. | | `button(етикет, onclick, вариант?, опции?)` | Кликаем компонент. | | `canvas(име, ш, в, опции?)` | Чертожна повърхност. | | `checkbox(име, етикет, опции?)` | Превключвател истина/неистина. | | `conjure` | Дефинира функция. | | `elevate(n, k)` | Закръгля `n` до `k` знака. | | `endwindow` | Затваря текущия прозорец. | | `escape` | Излиза от най-вътрешния цикъл. | | `essence` | Дефинира клас. | | `eternal` | Маркира неизменна стойност. | | `forge` | Декларира променлива. | | `group(заглавие) … endgroup` | Група компоненти с рамка. | | `heading(текст, ниво)` | Компонент за заглавие. | | `html(s)` | Вкарва санитизиран HTML в прозорец. | | `input(име, плейсхолдър, стойност, опции?)` | Едноредово поле. | | `json.parse(s) / json.stringify(v)` | JSON преобразуване. | | `map(име, шир, дълж, увеличение, опции?)` | Leaflet карта. | | `marker(карта, шир, дълж, етикет?)` | Добавя маркер. | | `matrix(редове, колони, fillFn)` | Създава матрица. | | `menu(заглавие) … endmenu`

`/ menuitem` | Меню бар. | | `paragraph(текст, опции?)` | Параграф. | | `progress(стойност, макс?)` | Лента за прогрес. | | `range(n)` | Списък 0..n-1. | | `readFile(път) / writeFile(път, съдържание)` | Файлов I/O. | | `row() … endrow / column() … endcolumn` | Оформления. | | `select(име, опции, опции?)` | Падащ селектор. | | `slider(име, мин, макс, стъпка?)` | Числов плъзгач. | | `speak(x)` | Извежда `x` в OUTPUT панела. | | `summon URL as име` | Импортира Gist модул. | | `table(заглавия, редове, опции?)` | Таблица с данни. | | `tabs() … endtabs / tab(t) … endtab` | Контейнер с табове. | | `textarea(име, редове, плейсхолдър, опции?)` | Многоредово поле. | | `toast(съобщение, вид?)` | Известие. | | `tome()` | Празен речник. | | `uiget(име) / uiset(име, стойност)` | Реактивно състояние. | | `window(заглавие, ш, в, опции?)` | Отваря прозорец. |

ГЛАВА 13

Приложение B — Индекс на уроците

Приложение В — Индекс на уроците

| # | Заглавие | Учи на | |---|----------|--------| | 1 | Здравей, прозорец | Първото ви приложение с прозорец. | | 2 | Калкулатор за бакшиш | Входни полета, плъзгачи и панел за резултат. | | 3 | Списък със задачи | Състояние, списъци, изтриване, запис в локално хранилище. | | 4 | Приложение за бележки | Textarea, няколко бележки, запис и зареждане през облак. | | 5 | Скицник за рисуване | Canvas, събития от мишка, избор на цвят, експорт като PNG. | | 6 | Помодоро таймер | Табове, лента за прогрес, аудио сигнали, настройки. | | 7 | Преглед на Markdown | Textarea + рендерирана визуализация едно до друго. | | 8 | Табло за времето | HTTP заявки, JSON, показване на отдалечени данни. | | 9 | Карта-изследовател | Leaflet с маркери, tooltips и поле за търсене. | | 10 | Мини електронна таблица | Таблици, редактиране на място, формули. | | 11 | Прозорец за чат | Скрол, async поток, балончета със съобщения. | | 12 | Файлов мениджър | Дърво с папки, drag & drop, облачна синхронизация. | | 13 | Калкулатор | Grid подредба, варианти на бутони, бързи клавиши. | | 14 | Игра с подскачаща топка | Canvas + анимационен цикъл със сблъсъци. | | 15 | Вашата първа библиотека | Пакетирайте код в Gist и го извикайте с `summon`. |

Колофон

Тази книга е генерирана от изходния репозитар на sdev на 2026-05-03. Основният текст е DejaVu Serif, заглавията — DejaVu Sans Bold, кодът — DejaVu Sans Mono. PDF се рендерира с ReportLab.